

Pulse

Design specification

A production-ready UI and UX specification for the Pulse fitness training app. Captured at the moment of design lock — every screen, every rationale, every decision.

Version	1.0
Date	May 2026
Status	Design-locked
Screens	35 documented
Audience	Eng · Design · Product

Why this document exists

This is the complete UI and UX specification for Pulse, captured at the moment of design lock and before implementation begins.

Two reasons this document is worth its weight. First, it preserves every design decision — colour values, layout choices, copy, behavioural rules — so that anyone implementing this app (a designer in Figma, a developer in Claude Code, a contractor picking up the work in three months) is working from one source of truth rather than reconstructing intent from a chat history. Second, it captures the rationale behind each decision, not just the decision. A button at 56px tall is meaningless without knowing it's that tall because the user's hand is sweaty mid-workout and 44px is the absolute floor for Apple's HIG. A subtitle reading "step 5 kg" is meaningless without knowing the increment is per-exercise and overridable, encoded in `user_exercise_preferences`. Decisions decay; rationale propagates.

The document is structured to be read either linearly (it tells a coherent story from the philosophy through the data model) or as a reference (the screen catalog is exhaustive and each screen entry is self-contained). Each of the 35 screens is presented with its visual mockup followed by the role, layout description, rationale, and data-model implications.

Contents

1. Product overview
2. Design philosophy and direction
3. [The Pulse design system](#)
4. [Universal Pulse idioms](#)
5. The smart logger — design IP
6. Batch 1 — Core lifter journey (screens 1-7)
7. Smart logger surface (screens 8-11)
8. [Logging edge cases \(screens 12-14\)](#)
9. [Logging surface completion \(screens 16-18\)](#)
10. Batch 2 — Progress and analytics (screens 19-24)
11. Batch 3 — Power features (screens 25-29)
12. [Batch 4 — Multi-tenant surfaces \(screens 30-35\)](#)
13. [Data model implications](#)
14. [Behavioural rules](#)
15. Patch 8 — agent file updates
16. What's next

Product overview

Pulse is a cross-platform iOS and Android fitness training app that competes with Hevy, Strong, Fitbod, JEFIT, and Gymshark Training. It serves three user types — lifters tracking their own training, personal trainers managing rosters of clients, and gym operators running facilities — from one codebase, with role-determined UI and RLS-determined data access.

The core promises are speed, offline reliability, and privacy. Logging a set takes one tap in the dominant case. The app works without signal. Personal data is private by default with explicit sharing seams. Apple Watch integration via a native SwiftUI companion app ships in a later phase but the schema is ready in v1.

The product is positioned in App Store category "Health & Fitness" — not "Medical." HIPAA does not apply; GDPR, MHMDA, and CCPA do, and every body-composition surface carries the "for personal fitness tracking only" disclaimer mandated by the fitness-domain-expert agent.

The tech stack underneath the design is React Native with Expo SDK 54+, NativeWind v4, Zustand and TanStack Query persisted via MMKV, FlashList for lists, Reanimated 4 for animation, and Hermes V1 with the New Architecture enabled. Sync is PowerSync over Supabase Postgres with Sync Streams (the new beta, not legacy YAML rules) and Clerk as the identity provider — Clerk Organizations carry the user-type and trainer-client-gym relationships, with PowerSync verifying Clerk JWTs via JWKS. Workers run on Fly.io or Railway with Node 22 LTS, BullMQ on Redis Cloud or Upstash Fixed (not PAYG — the rate dynamics break workouts in progress), and direct Postgres connections on port 5432 IPv4 (not Supavisor, due to its connection leak). Subscriptions are handled by RevenueCat from day one, with Superwall layered in at roughly \$25K MTR. Media is Mux with expo-video and expo-video-cache for iOS HLS offline. EAS Build, Sentry with Hermes source maps and UI Profiling enabled, PostHog EU for analytics and feature flags and session replay, and Postmark for transactional email round out the operational stack.

None of this is visible in the screens — that's by design — but it shapes what's possible. The smart logger commits to local SQLite via PowerSync's optimistic write path the moment a set is logged, with the network sync handled in the background. The QR scan flow uses signed JWTs verified locally before the gym context binds. Live Activities use the iOS-native expo-live-activity bridge. Each screen's UX assumes the underlying capability is there.

Design philosophy and direction

Two design directions were explored side-by-side at the start of the project. **Iron** was a dense, monochrome, almost utilitarian language inspired by Linear and Things 3 — serious, focused, made for the lifter who finds every other fitness app "fine but a bit busy." **Pulse** was a confident, premium, modern fitness aesthetic inspired by Whoop, the latest Strava direction, and Apple Fitness — larger numbers, more colour, more energy, still flat and disciplined.

Pulse was chosen as the canonical direction. The reasoning: it is the cleaner, more premium version of the broader fitness-app aesthetic and serves a wider market than Iron would. Iron was the more differentiated bet — there is no app that looks like Iron in the App Store's fitness category — but Pulse is the safer, broader, more emotionally resonant choice for a v1.

Pulse, as a design language, commits to a small number of strong choices.

The product is **dark by default**. The interior of a gym is darker than a sunlit room, screens at 9 PM are bright, and the colour palette is built around training intensity rather than productivity. Light mode is supported but is not the canonical view.

The accent is **one saturated training-green** — `#1D9E75` — used for exactly one element per screen.

The user's eye is trained, screen after screen, to find the next action in this colour. Inside the workout screen the accent is "Log set"; on home it's "Start workout"; on the routine builder it's "Save routine." This single-accent discipline is the most important visual decision in the entire spec.

Other colours appear only with **genuine semantic meaning**. Amber for personal records and warnings of the celebratory kind. Coral for caution. Light teal for body weight and chart fills. White, grey, and black are not present in the palette in any meaningful quantity.

Typography is **two weights only** — 500 medium and 400 regular. Numbers are **tabular** and tracked tighter at `letter-spacing: -0.02em` so they read as data, not as text. Labels above numbers are small (10–11px) and use the mid-teal `#5DCAA5`. The number always dominates the label.

Density is **between Apple Fitness and Hevy**. Card padding is 14px. Vertical rhythm between cards is 12px. Primary readouts run 32–40px (set logger, dashboard stats). Large headlines run 22–26px (dashboard "you're 14% stronger" pattern). Body text is 12–13px. Nothing is below 10px.

Cards have **rounded corners at 14px or 16px**. Buttons at 10–14px. Pills (tags, status indicators) at 6–8px. There is no shadow on any card — depth is communicated by colour layering against the page background.

Iconography is **Tabler outline** throughout. Outline icons feel lighter against the dark surfaces; filled icons feel heavy and obstruct the data. The single exception is `circle-check-filled` which is used to mark completed sets — a green dot is more readable than a green ring.

The fifth and last commitment is **language**. Pulse speaks to the user like a sportscaster. Headlines on the progress dashboard read "You're 14% stronger this quarter," not "Strength gain: +14%." Insight pills read "You're adding ~3 kg of est. 1RM per month. At this pace you'll hit 135 kg in ~3 weeks," not "Linear regression projects 135 kg in three weeks." Notifications read "12 days strong. 6 to beat your record," not "Streak: 12 days. Best: 18 days." Every piece of microcopy is the version a friend would say.

The Pulse design system

Surface stack

The Pulse interface is built on a four-tier surface model. Each tier sits on the one below it visually, and the colour values are tuned so that nesting deeper means a stronger green tint, not a darker shade.

TIER	COLOUR	ROLE
Page background	<code>#0A1410</code>	The deepest layer. The phone's screen. Nothing should match this.
Card surface	<code>#0E1F19</code>	The default card colour. Where most content lives.
Hero card	<code>#0F6E56</code>	The active context — "what the user is doing right now." One per screen.
Nested stat tile	<code>#08402F</code>	Used inside hero cards for label-above-number stat clusters.

Text on these surfaces is also tiered. On `#0A1410` and `#0E1F19`, headings are off-white `#E1F5EE`, labels are `#5DCAA5` mid-teal, and ambient text is `#9FE1CB` light teal. On `#0F6E56` (the hero), headings stay `#E1F5EE` but labels brighten to `#9FE1CB` for contrast.

Accent and semantic colours

The single canonical accent is `#1D9E75` saturated training-green. Text on this accent is always `#04342C` darkest-teal — the contrast ratio is comfortably above WCAG AA at any size, and it gives accent buttons a satisfying weight that pure white text wouldn't.

PURPOSE	COLOUR	WHERE IT APPEARS
Personal record / celebration	Amber <code>#FAC775</code> / <code>#412402</code>	PR badges, trophy icons, streak fire-flames
Warning / attention	Coral <code>#F0997B</code> / <code>#4A1B0C</code>	Stuck-lift detection, attention-needed clients, alerts
Positive delta / gain	Green <code>#97C459</code>	" <code>+12%</code> ", " <code>+25 kg</code> " arrows beside stats
Critical / destructive	Coral <code>#F0997B</code>	Delete account, disconnect integration

Typography

The product uses a single font family — system default (San Francisco on iOS, Roboto on Android) — at two weights only. Medium 500 and Regular 400.

TYPE	SIZE	WEIGHT	USE
Display number	32–40px	500	Primary set logger, weight readout, daily stats
Hero number	22–28px	500	Dashboard headlines, body weight
Title	18–22px	500	Screen titles
Subtitle	14–16px	500	Card headers, exercise names
Body	11–13px	400	Captions, secondary text
Label	9–11px	400	Above-number labels, section headers (uppercase)

Spacing, corners, iconography

Card outer padding is 12–14px; card stack gaps 8–12px; section spacing 14–18px. Element minimum tap size is 44px (Apple HIG floor). Primary action height runs 50–56px; secondary 36–42px.

Cards round at 14–16px, buttons 10–14px, pills 6–8px. Borders are 0.5px hairline in `#08402F` for most card outlines, escalating to 1.5px `#5DCAA5` for selected or active elements. No shadows anywhere — depth is communicated by colour layering.

Tabler outline icons at 14–22px throughout. Key icons: `ti-barbell` (workout), `ti-trophy` (PR), `ti-flame` (streak), `ti-clock` (rest), `ti-trending-up` (improvement), `ti-link` (superset), `ti-circle-check-filled` (completed set), `ti-shield-check` (verified gym sticker), `ti-qrcode` (scan equipment), `ti-bolt` (premium).

Universal Pulse idioms

Six visual patterns repeat across the catalog. Recognising these makes the rest of the spec easy to implement consistently.

1. The hero card

A card with background `#0F6E56`, padding 14–16px, and a small uppercase 10–11px label at the top in `#9FE1CB` mid-teal. Below the label comes the title (17–22px, weight 500, `#E1F5EE`), then either action content or a row of nested stat tiles. The hero card represents *what the user is doing right now* — the current exercise mid-workout, today's planned routine on home, the currently-selected gym in settings. There is at most one hero card per screen.

2. The nested stat tile

A small tile with background `#08402F`, padding 8–10px, used inside hero cards for "label-above-number" stat clusters. The label is 9–10px in `#9FE1CB`, the number is 14–16px weight 500 in `#E1F5EE`. Stat tiles are arranged in groups of 2–4 across the bottom of a hero card.

3. The stepped input pair

The canonical numeric input. Two columns, each with a label (WEIGHT / REPS), the current value as a large number (24–32px), and $-/+$ buttons either side. The user does not see the system keyboard; the stepper is the input method. Each column has a "step" footer ("step 5 kg", "step 1 rep") that documents the increment for this exercise. The increment is per-exercise and overridable via the `user_exercise_preferences` table.

4. The single accent button

The primary CTA on every screen, rendered at 50–60px height with `#1D9E75` background and `#04342C` text, weight 500, label specific to the action ("Log 80 × 8", not "Save"; "Log and move to B", not "Next"). At most one accent button is visible at any moment.

5. The dashed border with amber chip

The universal "draft" or "unsaved" signal, used by the implicit-confirm pattern. A card transitions from a solid 0.5px `#08402F` border to a 1.5px dashed `#5DCAA5` border, and a small amber chip ("unsaved") appears in the top-right corner. This is unique to this state — no other element in the system uses dashed borders.

6. The chevron-left header with breadcrumb subtitle

The universal header for any stack screen, with a `ti-chevron-left` at 20–22px on the left, a context block in the centre (a small 10–11px breadcrumb in `#5DCAA5` ambient teal showing the parent context, and a 12–14px subtitle showing the current screen state), and an optional secondary action on the right.

The smart logger — design IP

The smart logger is the single most important innovation in this app, and the reason a serious lifter would switch from Hevy or Strong. It is built around four principles, each of which is encoded in specific UI behaviour.

Principle 1 — Match last session by default

The dominant case for any working set is "do what you did last time, plus or minus." This is true for somewhere between 70 and 85 percent of all sets in any structured strength program. The app should

treat that case as the default, not as a configuration the user has to opt into.

When the user taps "Begin first set" on an exercise, the weight and reps input is pre-filled with the values from their last completed set of that exercise on this equipment. The big primary button reads "Log 80 × 8" (or whatever the last values were) — not "Log set." Tapping it commits a new set with those exact values and advances the user to the next set or exercise. This is the **one-tap path**, and the friction budget for this path is exactly 1 tap.

Principle 2 — Stepped, contextual adjustment

If the user wants to go heavier or lighter than last session, the next-most-common adjustment is by one increment — one plate, one pin, one small dumbbell jump. The system encodes these increments per exercise.

EXERCISE CATEGORY	DEFAULT INCREMENT	NOTE
Barbell compound	±5 kg	Long-press: ±2.5 kg
Selectorized cable machine	±1 pin	Approximate kg shown in muted text
Cable / pulley (kg stack)	±5 kg	
Dumbbells	±2.5 kg	Reflects how dumbbell racks step
Bodyweight + load	±1.25 kg	Weighted vests, dipping belts
Isolation barbell	±2.5 kg	

These defaults are encoded in the catalog. Each user can override per exercise in `user_exercise_preferences`. For machines with pin-based selectorized weight stacks, the primary readout is the *pin position*, not the kilogram value. The user remembers "pin 9," not "65 kg."

Principle 3 — Implicit confirm

Hevy and Strong both require the user to actively confirm each set with a checkbox or a +1-set tap. This is friction. It is also the wrong mental model — the user *did* the set in the gym; they're not asking permission to record it, they're notifying the system that it happened.

Every set is provisional until the user either taps "Log" or starts a new set, at which point the previous set's last-shown values become canonical. Moving to a different exercise without confirming discards the in-progress set with a small undo affordance.

The implicit-confirm pattern saves an entire tap per set across the entire app, which over a 21-set workout is 21 taps the user didn't have to make. In aggregate this is the difference between "this app feels fast" and "this app feels like Hevy."

Principle 4 — Per-equipment binding

The user does not "do bench press." They "do bench press on Bench 3 at Iron Hub Belgrade." The leverage of one machine is different from another. The cable run on a selectorized stack is different across manufacturers. The user's perception of progress is anchored to the specific machine.

Every logged set carries three foreign keys: `exercise_id`, `gym_id`, and `gym_equipment_instance_id`. Last-session matching, PR detection, and history all default to the per-equipment scope when the user is at the same gym, and fall back to per-exercise across gyms when the user is travelling.

Edge case — supersets

A superset is two or more exercises performed back-to-back with no rest between, then a longer rest after the round, then repeat. The user thinks of this as one unit ("3 rounds of pull-ups + dumbbell rows"), not as two independent exercises.

The data model encodes this with a `set_group` row that ties multiple `workout_sets` together. The `set_group` has a `kind` field set to `superset`, and stores the round number. Each `workout_set` in the group gets a `set_group_id` and a `set_group_position` (0 for A, 1 for B, etc.). The rest timer is associated with the *group*, not the individual sets — it suppresses between A and B, and fires after both are logged.

Edge case — drop sets

A drop set is one set followed immediately by lighter weight and continued reps, often two or three drops in sequence. The user thinks of this as one extended set with multiple weights, not three separate sets.

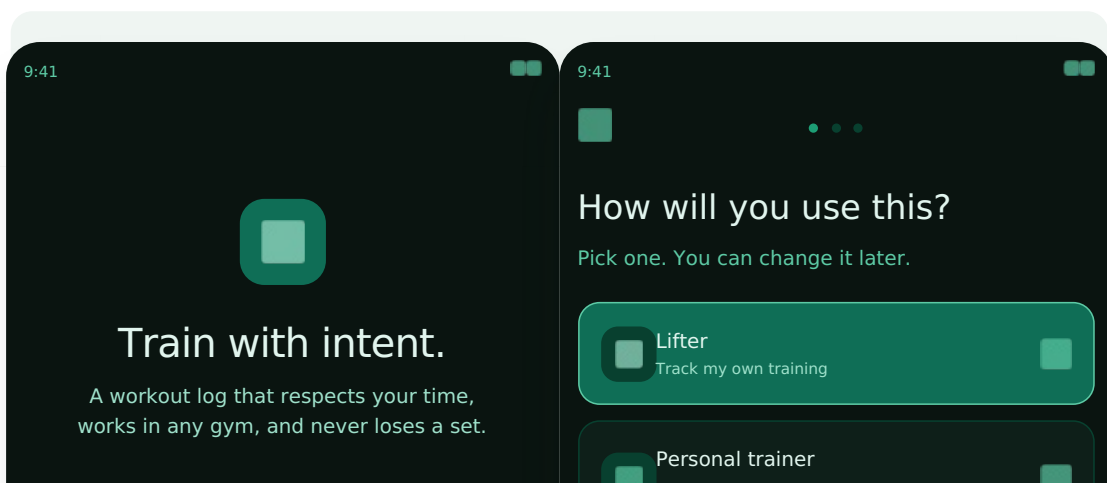
The same `set_group` pattern handles this with `kind = drop_set`. Position 0 is the top set, positions 1, 2, 3... are the drops. Rest happens only after the last drop. PR detection on drop sets is interesting: traditional PRs on a drop set's top weight are detected normally, but the app also detects "drop PRs" — improvement on the same drop weight versus last time. A small celebratory pill ("+4 reps on drop 1 vs last time") appears in the summary, providing emotional payoff for the harder work the user actually did.

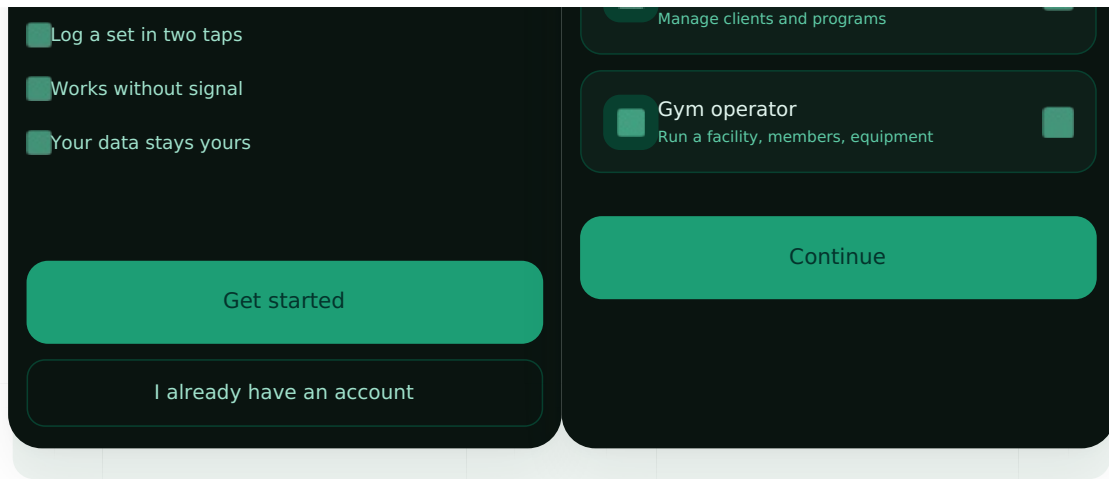
BATCH 1

Core lifter journey

The seven foundational screens of the lifter experience. These cover everything from cold launch through first workout completion.

Screen 1 — Welcome and user-type selection





ROLE

The first 60 seconds of the app, broken into two sequential screens. Welcome introduces the product's three pillars. User-type selection determines the Clerk Organization the user joins, the sync streams they get, and the UI variant they see for the rest of their session.

RATIONALE

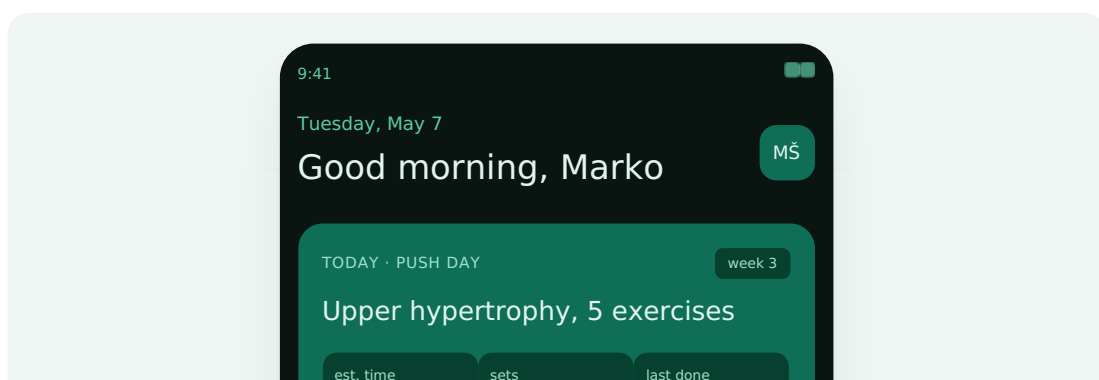
User-type selection is the most consequential decision in the entire app — it changes the schema visible to the user, the RLS policies enforced server-side, and the UI variant rendered. So it earns its own screen rather than being buried in a dropdown deep in settings.

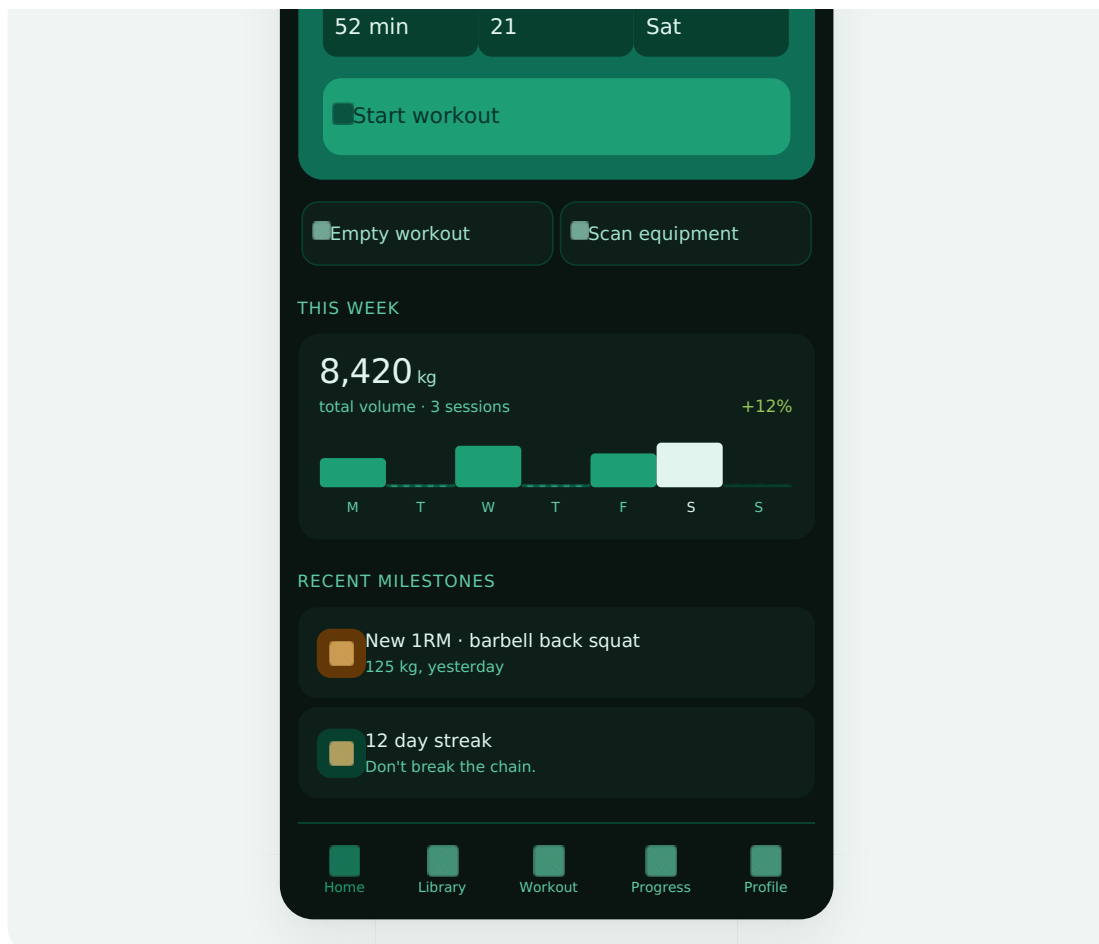
The welcome screen's three pillars are not random marketing fluff. Each one is a real promise tied to a real feature, and each one is the answer to a different competitor pain point: Hevy is slow for power users (we promise two taps); Strong and Fitbod struggle in basement gyms with no signal (we promise offline); every competitor sells data in some form (we promise private-by-default). These pillars also become the App Store screenshot captions.

DATA AND BEHAVIOURAL IMPLICATIONS

The user-type selection is persisted in the user's Clerk public metadata as `userType`, and the Clerk Organization is auto-created (for trainer and gym_operator) or auto-joined (for lifter) immediately on confirmation. PowerSync sync streams are activated based on this value: `user_data` (always), `catalog` (always), `trainer_clients` (trainers), `gym_aggregates` (gym operators).

Screen 2 — Home





ROLE

The default landing surface for the lifter. Tells the user what to do next without making them think — a primary "Start" action, today's planned routine surfaced if it exists, glance at this week's training cadence, recent achievements.

RATIONALE

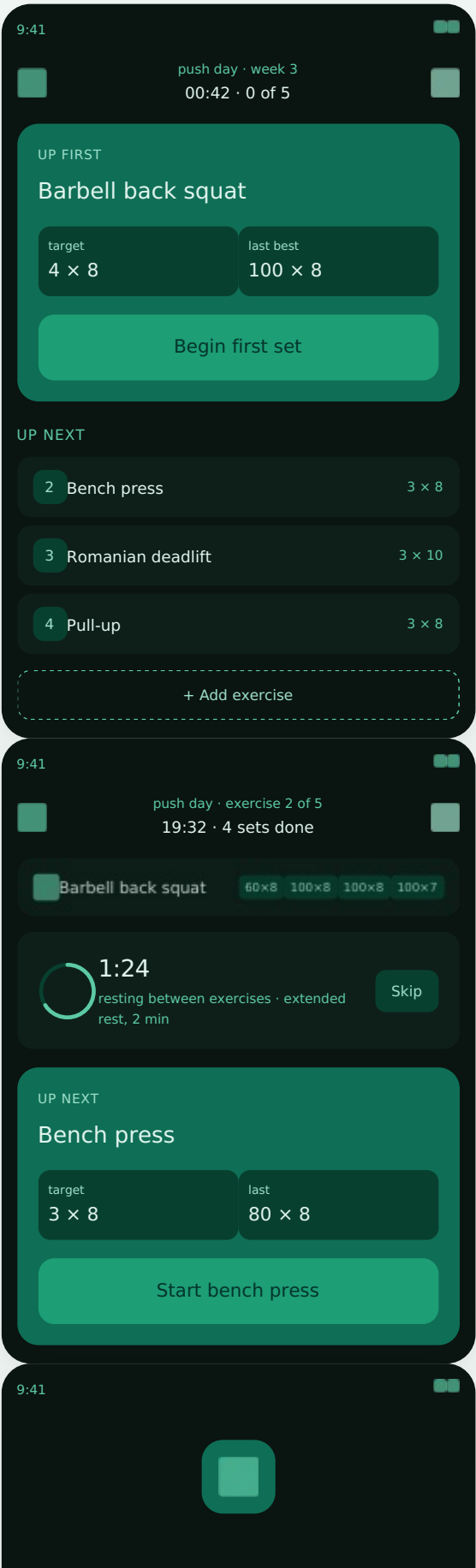
The home screen's job is "tell me what to do next." Today's planned routine occupies the largest card with the single accent button — eyes find it instantly. The two secondary buttons handle the off-plan cases without competing for attention. The this-week histogram is glanceable; the user sees their training cadence in 0.2 seconds and either feels good (lots of green bars) or has a soft motivator to train today (gaps in the histogram). Recent milestones are celebratory but not actions — they live below the fold to reward scrolling.

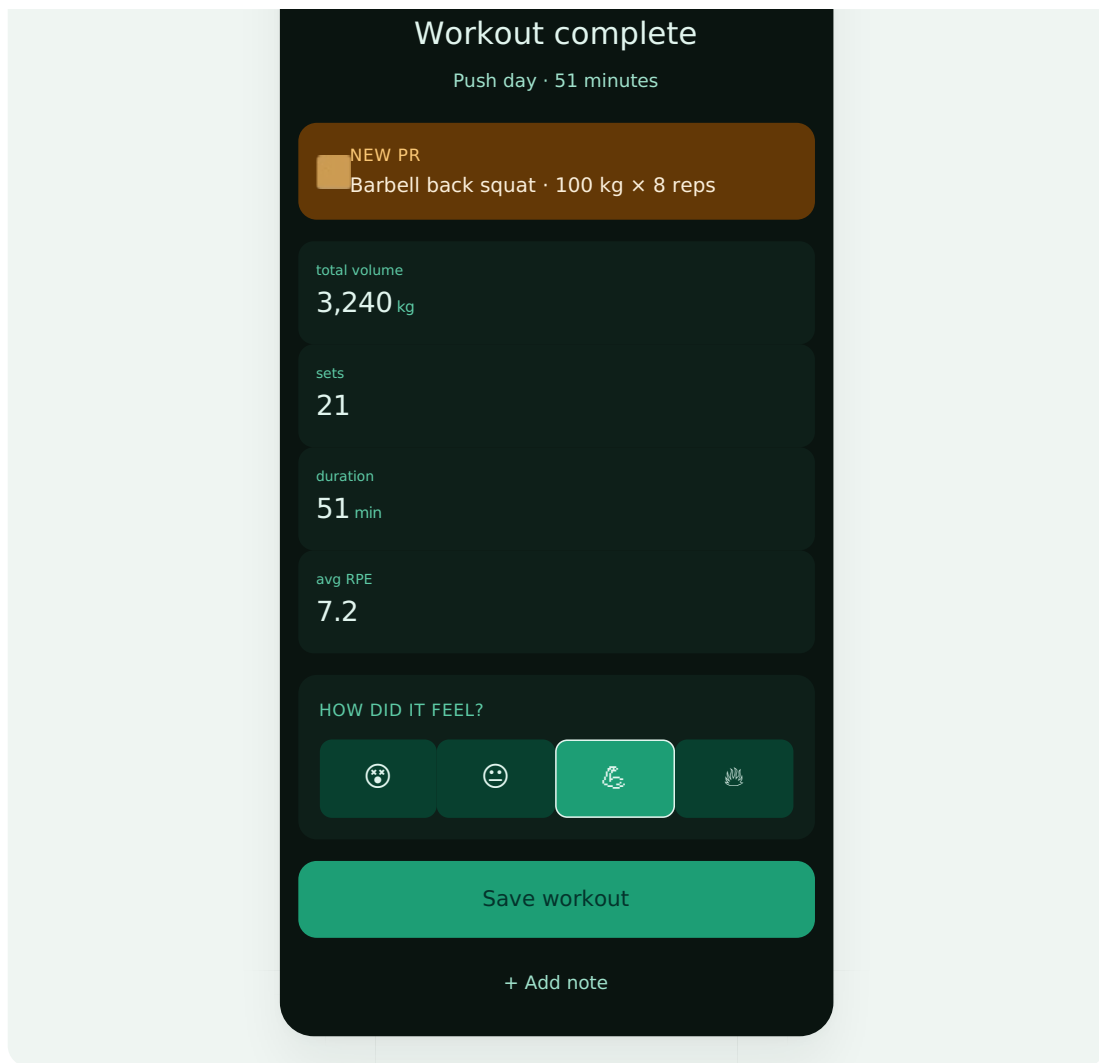
We deliberately omit a "social feed" and an "explore" tab. Pulse is private by default; we don't have a feed. Pulse trusts the user to know what they want; we don't need to push explore content at them.

DATA AND BEHAVIOURAL IMPLICATIONS

The "today's planned routine" card surfaces the next scheduled session from a `scheduled_workouts` table joined against the user's active program. The 7-bar histogram queries `workout_sets` grouped by `local_date` (using the user's timezone, not UTC) and sums volume per day. The aggregation is local-first via PowerSync; the query runs against the local SQLite directly with no network round-trip. The "Recent milestones" rows pull from a `personal_records` view that updates on each set commit via a server-side trigger.

Screen 3 — Active workout, three states





ROLE

The screen the user spends 60–90 minutes per day inside. If this screen is wrong, the app fails regardless of how the rest looks. Three states documented: just started (no sets logged yet, exercise picker visible), between exercises (collapsed view of completed exercise, next one queued), and post-workout summary.

RATIONALE

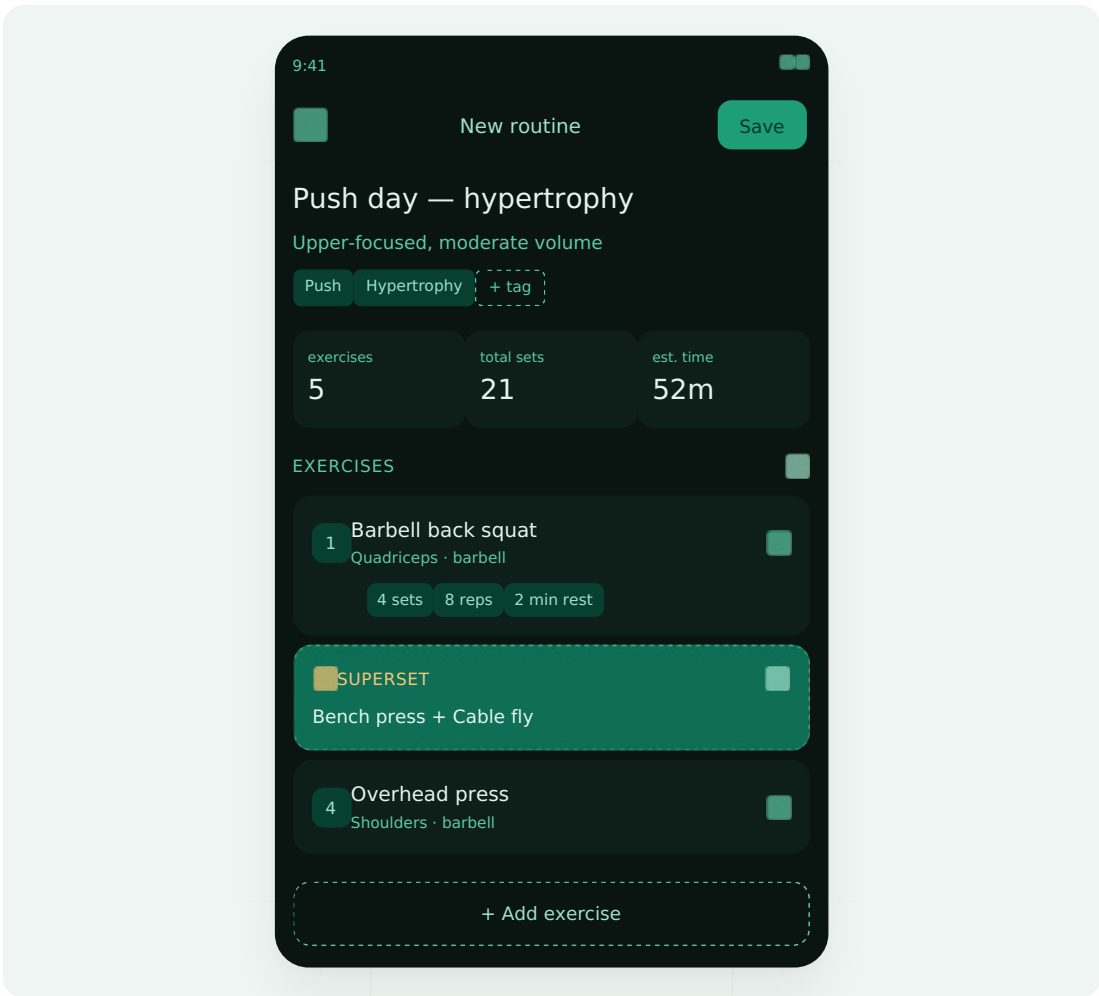
The three states encode the natural arc of a workout — beginning, middle, end. The "between exercises" state collapses the completed exercise to one line precisely because the user does not need to dwell on what they just did — they need to focus on what's next. The four set-chips on the collapsed row give a 0.2-second glance summary.

The completion summary leads with the PR badge (when one exists) because that's the emotional payoff of the session. The how-did-it-feel emoji row is the soft RPE capture — it doesn't ask the user to think about RPE in numbers, just to pick a face. The mapping to a 1–10 RPE happens server-side in the analytics layer.

DATA AND BEHAVIOURAL IMPLICATIONS

The workout session is a single `workout_session` row. The "Save workout" action sets `ended_at` and runs PR detection in a single transaction. PR detection compares the new sets against the user's history per `exercise_id` and per `gym_equipment_instance_id`, creating `personal_record` rows for any new records.

Screen 4 — Routine builder



ROLE

The screen serious users live in when not actively training. The job: let a user assemble a multi-exercise routine fast, see what they've built, save it. The output is a `routine` row with N `routine_exercises` rows in order.

RATIONALE

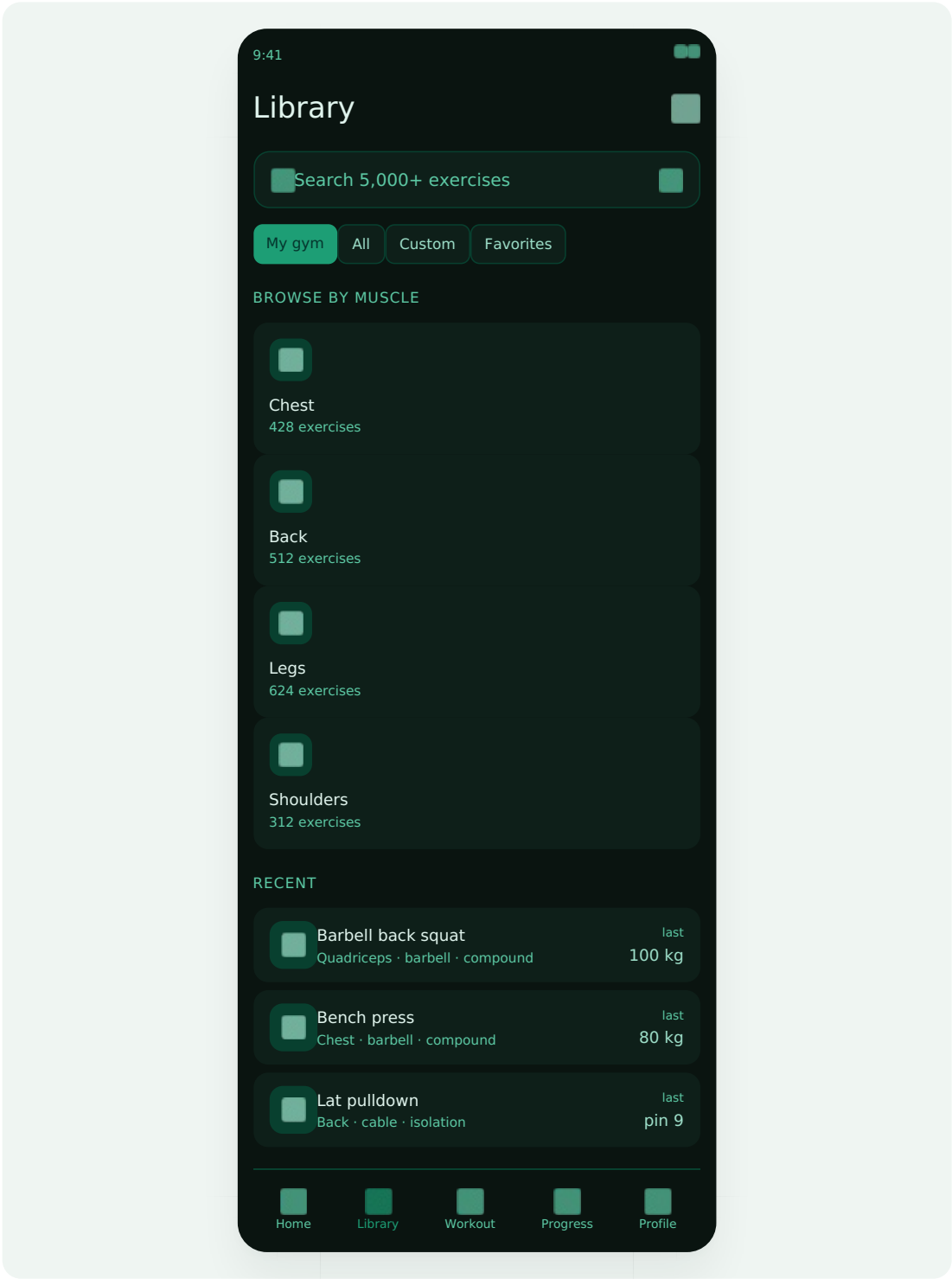
The inline editable title is a small but meaningful choice — most routine builders make the user navigate to a separate "edit name" field. Making it inline says "this routine is yours, not the system's." The dashed border on the +tag and +exercise buttons is a Pulse convention — dashed borders mean "draft or extensible," solid borders mean "committed."

The three-tile summary updating live is the trust-building moment: the user sees "5 exercises · 21 sets · 52 min" and knows what they're building. No "calculate" button. The superset card is visually distinct (saturated green background) because it's a structural element, not just another exercise.

DATA AND BEHAVIOURAL IMPLICATIONS

A `routine` row holds the routine metadata. `routine_exercises` rows are ordered by a `position` integer. Supersets are encoded as a `routine_exercise` row with `kind = superset` and a `superset_group_id` matching the entries of the partnered exercises. Inline edits use the implicit-confirm pattern — typing in the title field doesn't trigger a save, but tapping "Save" in the header does.

Screen 5 — Exercise library



ROLE

The library is where users browse to add an exercise to a routine or to start an ad-hoc workout. Filtering by current gym is the default. Search is primary; categories are secondary.

RATIONALE

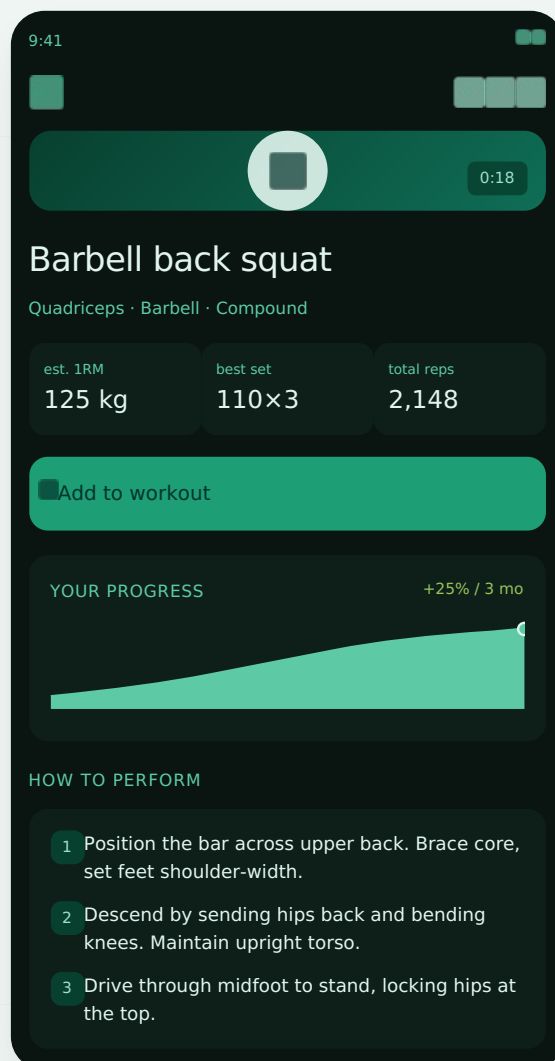
The "My gym" default filter solves the biggest competitor pain point — Hevy and Fitbod both show you a Smith machine you don't have access to. By filtering to the current gym's equipment by default, we cut the noise to about a third of the catalog and the user sees only what's actually possible.

The last-weight indicator on each recent row is what turns the library from "exercise catalog" into "your training memory." When the user taps "Bench press" and sees "last 80 kg" right there, they walk into the gym already knowing what they're going to lift.

DATA AND BEHAVIOURAL IMPLICATIONS

The library's filtering is a join: `exercises` left-joined to `gym_equipment` where `gym_id = current_gym_id`. The exercise catalog comes from ExerciseDB (5,000+ exercises, chosen over wger because of higher-quality video coverage and proper IP rights). Custom exercises are user-created rows scoped by `user_id`.

Screen 6 — Exercise detail



Tapping any exercise opens this modal. The user can preview the exercise via video, see their personal history, read instructions, and either add it to a routine or start logging it directly.

RATIONALE

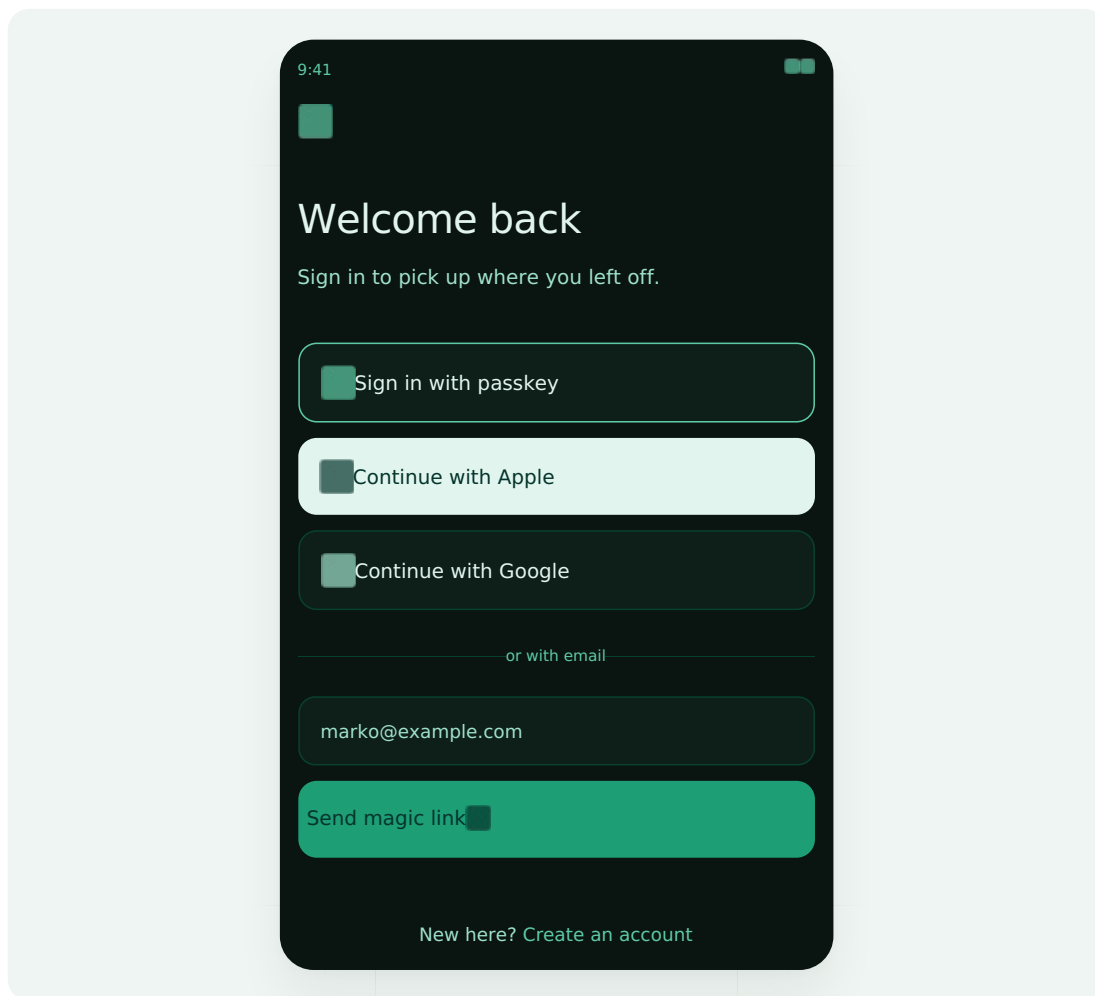
Video at the top because that's what users want to see first — "is this the exercise I think it is?" The three personal stats (1RM, best set, total reps) are *the user's numbers*, not generic facts. This is what makes the detail screen feel personalised. A novice sees zeros and is invited to set them. A power user sees their entire history at a glance.

The 3-month sparkline is the smallest progress chart in the app — it's a teaser. Tapping the card opens the full exercise progress chart (screen 21), which is the deep-dive view.

DATA AND BEHAVIOURAL IMPLICATIONS

The video URL is a Mux playback ID resolved from the `exercise.video_id` field. The `expo-video` player streams via HLS for adaptive quality, and `expo-video-cache` ensures pre-cached videos play instantly from disk. The three personal stats query `workout_sets` for this user and exercise: 1RM est. uses Epley formula, best set picks the highest weight × reps product, total reps sums across all logged sets.

Screen 7 — Sign-in



ROLE

The Clerk-rendered authentication screen, styled to match Pulse. Passkey is the primary method. Apple and Google sign-in are secondary. Email magic-link is the fallback.

RATIONALE

Passkey is the future-default in 2026 and we want users to set one up. The bordered card signals "this is the recommended method" without being pushy. Apple's button has mandatory styling per Apple's HIG when other social providers are present — white background, black text, Apple logo. We can't restyle it without risking App Review issues.

Email magic-link is below the divider because in 2026 it's the third-best path. We support it (forever, given many users still prefer email) but we don't promote it.

DATA AND BEHAVIOURAL IMPLICATIONS

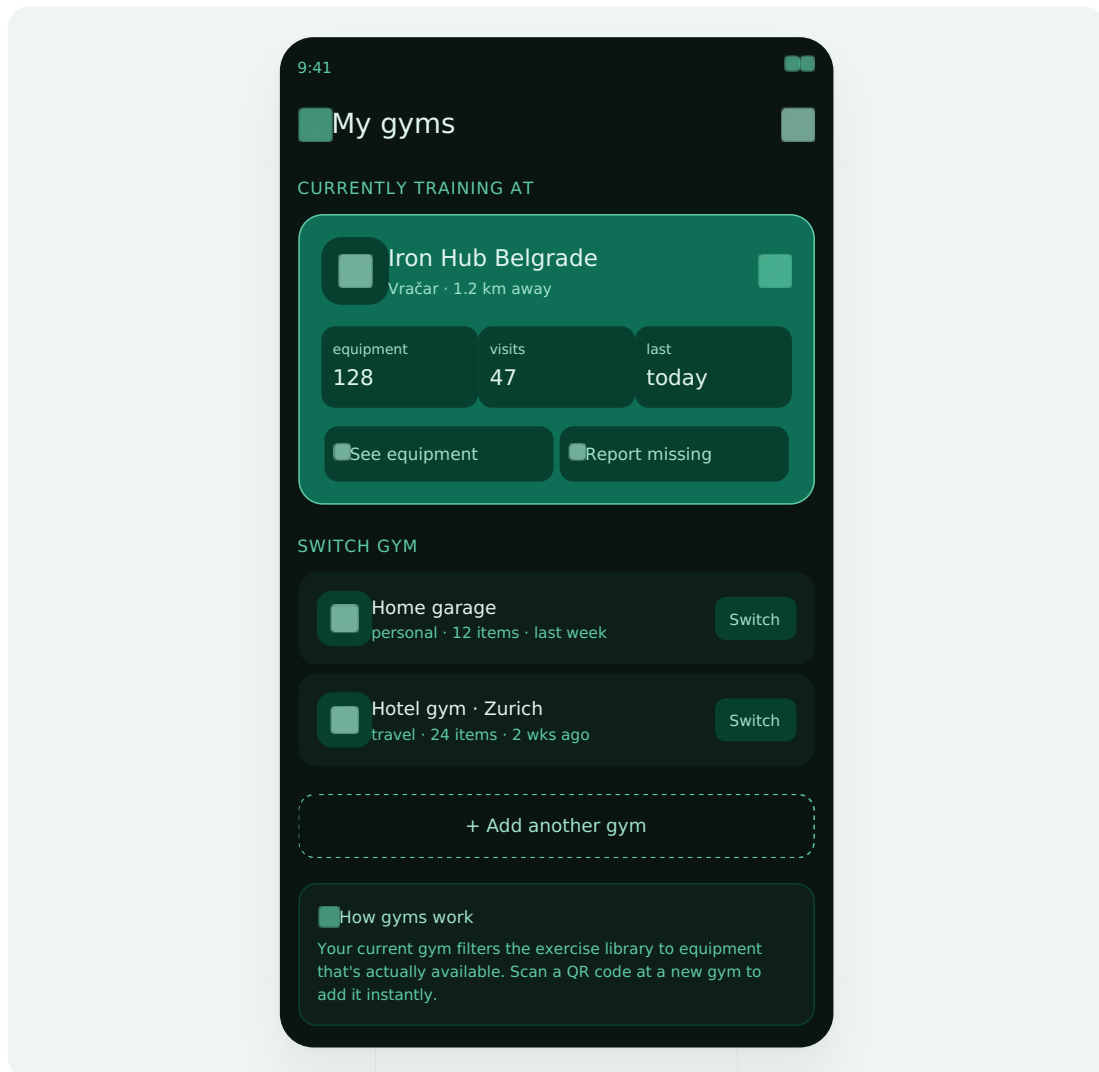
Clerk handles the entire authentication flow. On successful sign-in, Clerk issues a JWT that PowerSync's edge function validates via JWKS. The user's `userType` is read from Clerk's public metadata, which determines the sync streams that activate. The "passkey is recommended" framing also serves a real security goal — passkeys are phishing-resistant by design, and they let us drop password-based auth entirely.

SECTION

Smart logger surface

Four screens that encode the gym-context filtering and the smart logging innovations. These are the heart of the product's differentiation.

Screen 8 — Gym selection in settings



ROLE

The user opens this from Profile → My gyms. They see every gym they've checked into, with one marked "current." Tapping a different gym makes it current; the library, exercise selector, and pre-flight check reshape immediately.

RATIONALE

Current gym is the hero so the user's mental model is anchored. Three at-a-glance stats prove the binding is meaningful — 128 equipment items associated with this gym, 47 visits logged, last visited today. These are facts the user can verify against their reality.

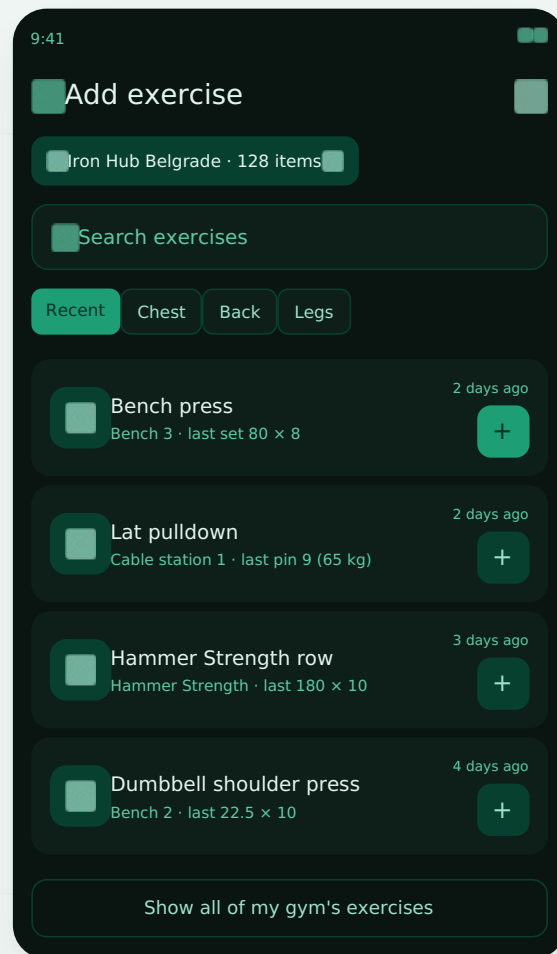
The "Switch" buttons are deliberately small (28px tall, brief 11px text) so they read as quiet

alternatives. The current-gym card dominates because that's the gym the user is at right now. The "Report missing" button is the user-generated equipment-fix path — small but important growth loop.

DATA AND BEHAVIOURAL IMPLICATIONS

A user belongs to many gyms via `user_gyms` (a many-to-many table). The "current gym" is a single `current_gym_id` foreign key on the user's profile. Switching is one write that triggers a reactive recompute of the library filter and recent-exercise sorts.

Screen 9 — Smart exercise selector



ROLE

What the user sees when they tap "Empty workout" or "Add exercise" mid-session. Filtered to the current gym's equipment by default, sorted by what the user used last time, with last-weight and last-equipment-instance shown inline.

RATIONALE

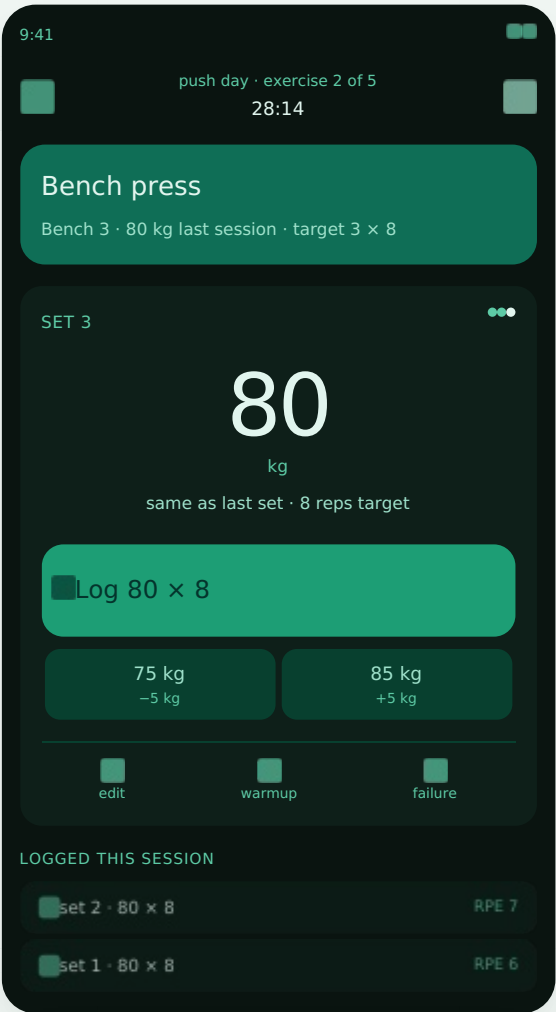
The first row's + button is accent green because it's the most-likely-next-exercise. This is anticipatory UX — the app makes a best guess and renders it as the default action. The user can still tap any other row, but the eye is drawn to the freshest option.

Each row shows the *specific equipment instance* (e.g., "Power rack 2", "Bench 3", "Cable station 1"), not just the equipment category. This is the per-equipment binding paying off in the UI. The lat pulldown row showing "pin 9 (65 kg)" rather than just "65 kg" matches how the lifter actually thinks.

DATA AND BEHAVIOURAL IMPLICATIONS

The query joins `exercises` → `gym_equipment` → `user_workout_sets`. The sort key is `MAX(workout_sets.created_at)` per exercise. The "last" annotation pulls the most recent set's weight + equipment_instance + relative time.

Screen 10 — Smart set logger, two equipment models



Progress tracks pin position.

ROLE

The screen that replaces everything. The user has tapped an exercise and is logging a set. Two equipment models are shown side-by-side: barbell bench (± 5 kg increment) and selectorized cable (± 1 pin increment).

RATIONALE

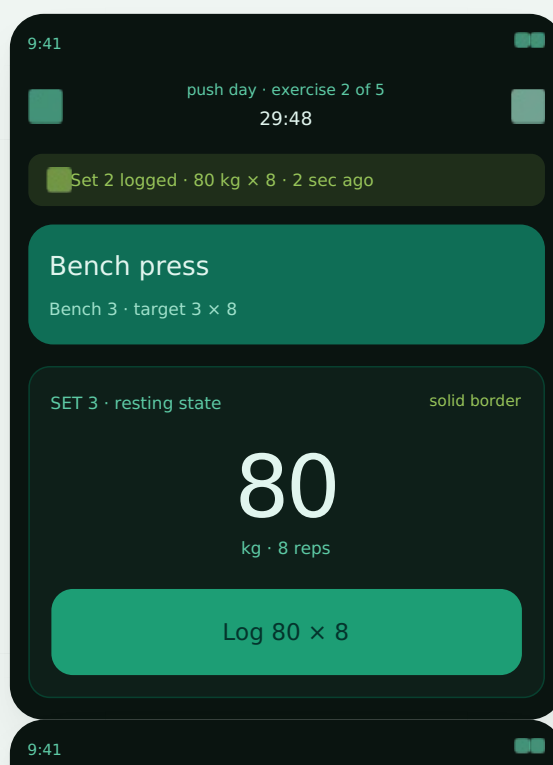
Two side-by-side examples make it visually obvious that the increment logic is per-exercise, not one-size-fits-all. The barbell shows ± 5 kg with kg-only readout. The cable shows ± 1 pin with kg-as-supplementary-info.

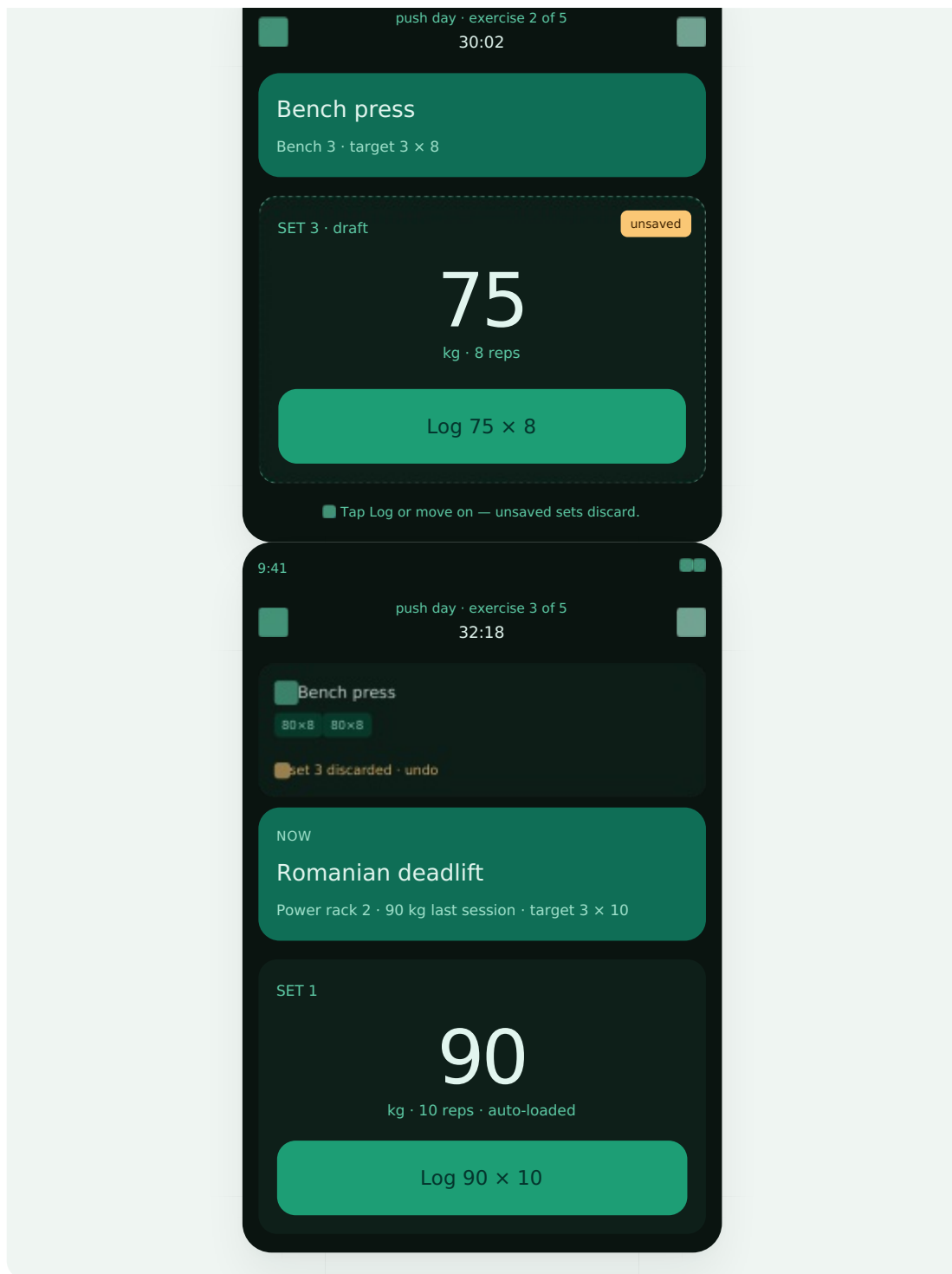
The button label is contextual to the action ("Log 80 $\times$ 8" / "Log pin 9 $\times$ 10") rather than generic ("Save", "Log set"). The user reads what's about to happen, not what control they're pressing. The footer tertiary actions ("warmup", "failure", "drop set", "edit") expose the rare-but-real cases without crowding the primary surface.

DATA AND BEHAVIOURAL IMPLICATIONS

The set commit writes a `workout_set` row with `weight`, `reps`, `exercise_id`, `gym_equipment_instance_id`, `workout_session_id`, `pin_position` (for selectorized), and metadata flags. For selectorized machines, the canonical stored value is `pin_position`. The `weight` field is filled with the approximate kg conversion for the machine, and PR detection runs on `pin_position` for selectorized exercises rather than `weight`.

Screen 11 — Implicit-confirm sequence





ROLE

Three sequential frames showing the implicit-confirm pattern in action. Each frame is "what the screen looks like right after the user's most recent tap."

RATIONALE

The three-frame sequence is the most important UX explanation in the entire spec. It documents a behaviour that competitors (Hevy, Strong, Fitbod) get wrong, and that lifters notice immediately when it's right.

The key visual signal is the **dashed border + amber chip**, which is the universal "draft" indicator. The user can read at a glance: solid border = confirmed, dashed border + amber chip = unsaved.

The undo affordance on discard is small but essential. Silent data loss is hostile, even when

the discard was the user's intent. The amber-coloured note in the previous exercise's summary is unobtrusive but findable.

DATA AND BEHAVIOURAL IMPLICATIONS

The "drafting" state lives only in the local workout store (Zustand + MMKV). It is *not* persisted to SQLite or synced. Only confirmed sets become `workout_set` rows. Draft state is in-memory.

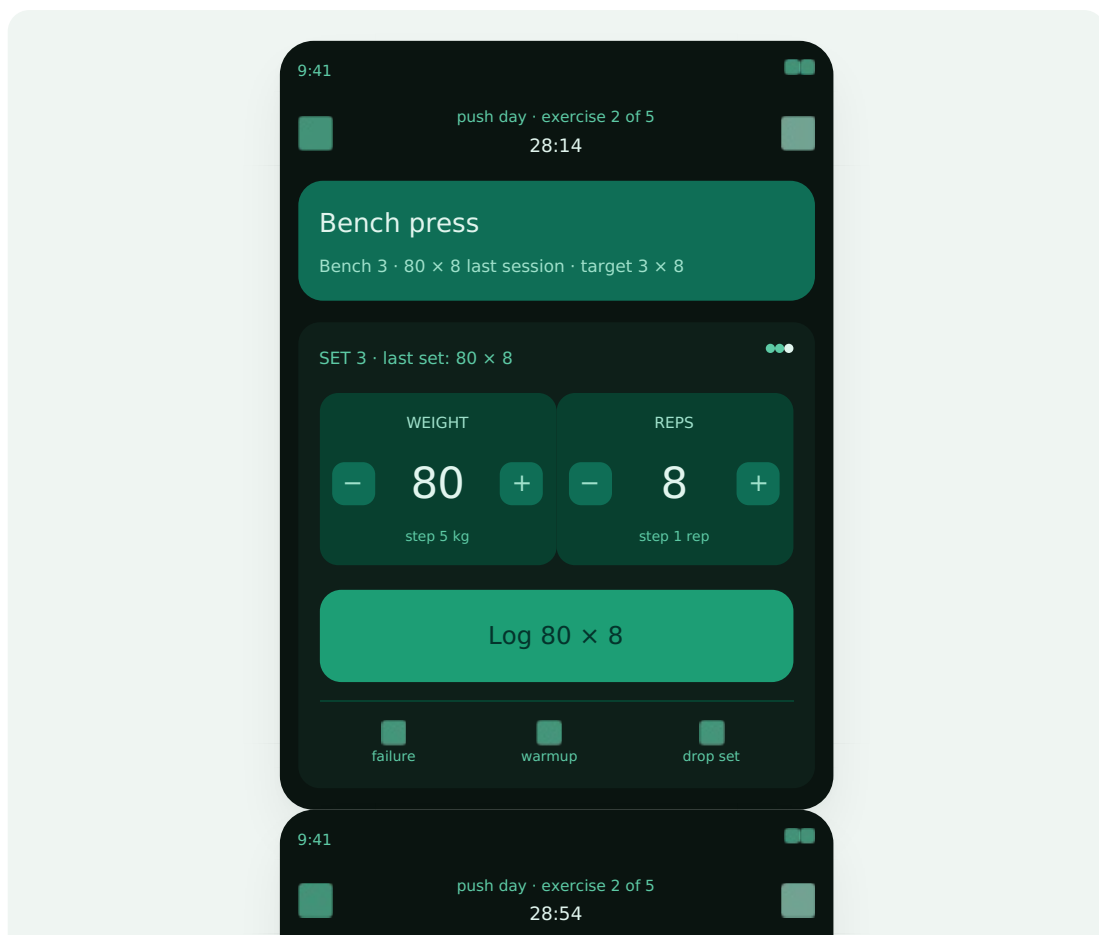
The "undo" action retrieves the last draft from a short-lived in-memory ring buffer (last 5 drafts). Drafts older than 5 minutes or beyond the buffer are not recoverable — the user has moved on. The "after 3 confirmations, hide the hint" rule is tracked in `user_preferences.has_seen_implicit_confirm_hint`.

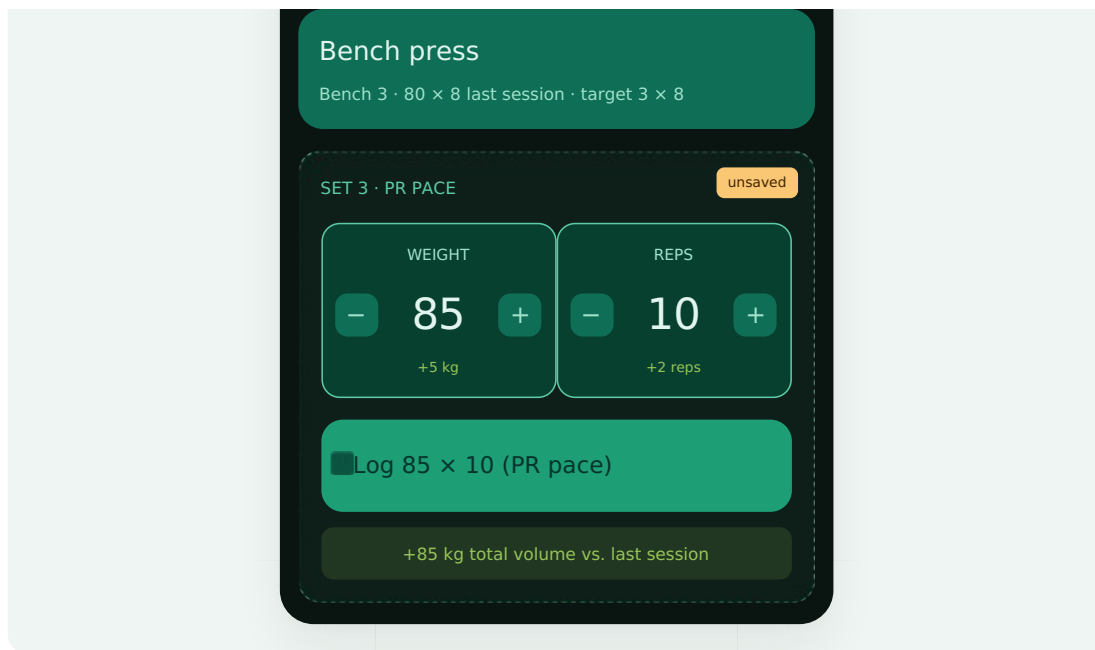
SECTION

Logging edge cases

Three screens for the parts of training that break naive set trackers — stepped reps, supersets, and drop sets.

Screen 12 — Stepped reps logger





ROLE

The set logger with both weight AND reps as $\pm$ -stepped values. This is the final form of the smart logger from screen 10, with reps also defaulting to last session's value rather than blank-waiting-for-input.

RATIONALE

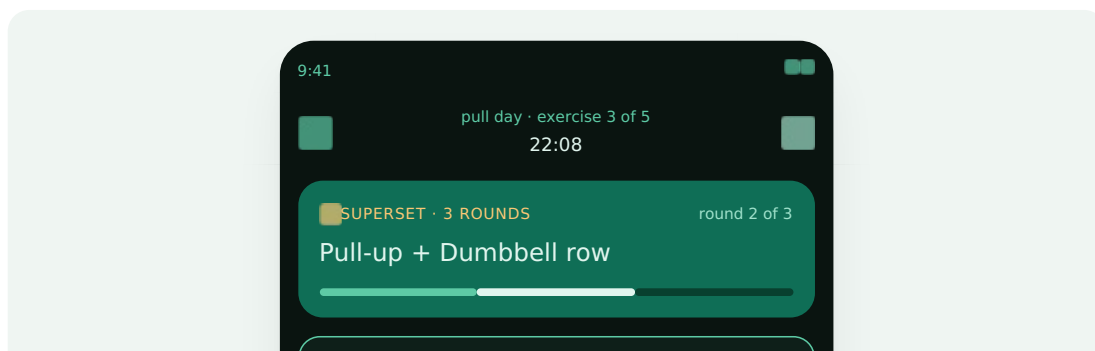
Stepped reps with last-session-matching makes the workflow consistent — both weight and reps treat the past as the default and adjustments as the exception. The user mental model is "match unless I'm pushing today."

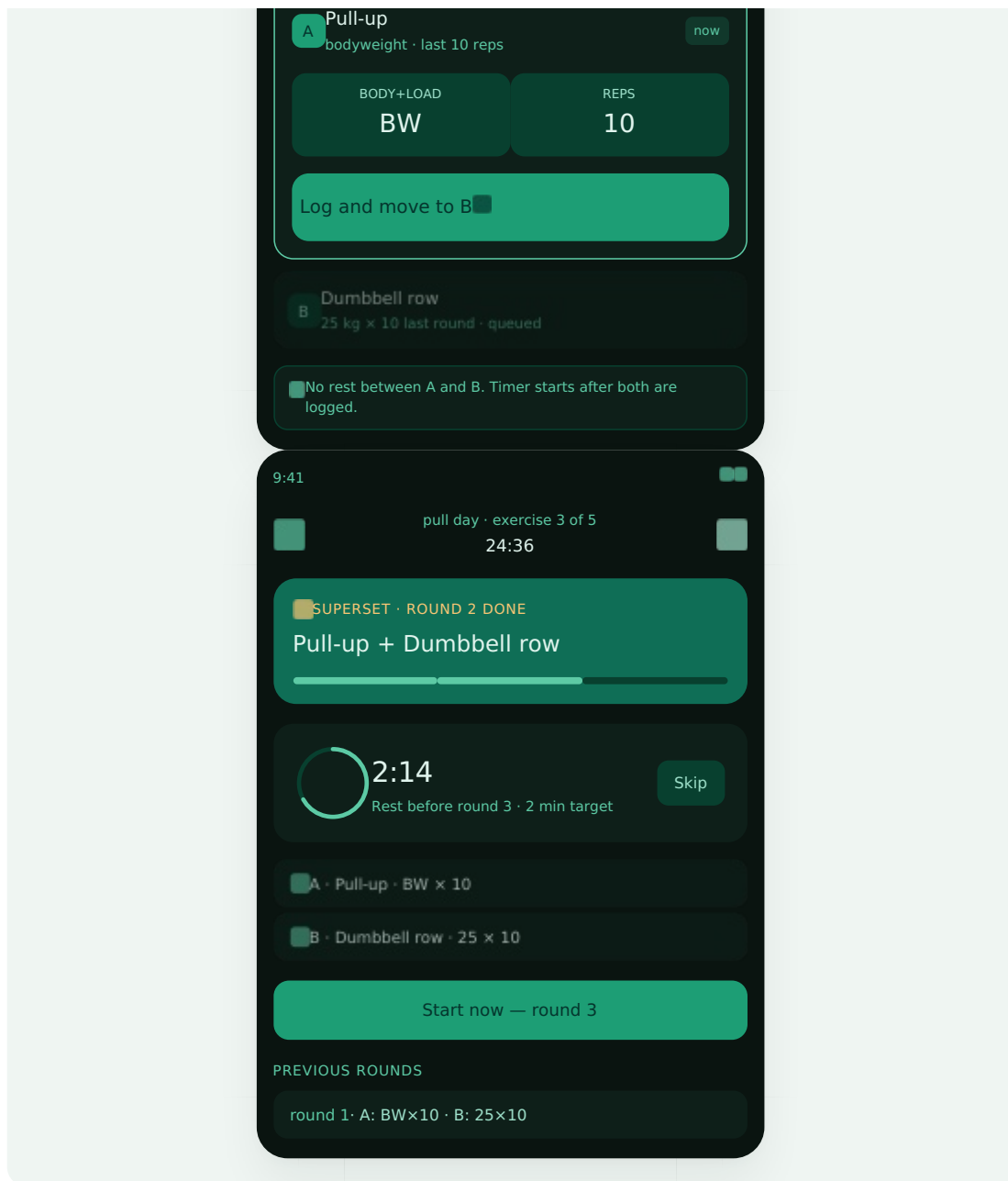
The PR-pace anticipation is the differentiating moment. When the user is *about to* hit a PR (not after the set, but in the draft state), the app recognises it and changes the button. This is forward-looking celebration — the user knows they're about to do something special.

DATA AND BEHAVIOURAL IMPLICATIONS

PR-pace detection is computed live in the draft state. The pace check compares the draft values against the user's stored PRs for this `exercise_id` × `gym_equipment_instance_id`. If any of 1RM-pace, reps-pace, or volume-pace evaluates true, the button transforms.

Screen 13 — Superset logger





ROLE

Supersets rendered as one round with two exercises stacked. Two frames documented: round 2 first-exercise active, and round 2 done resting.

RATIONALE

The chain-link icon + "SUPERSET" label is the universal signal that this is a linked structure. The round-progress bar shows where the user is in the sequence at a glance. The "now" pill and 1.5px green border on exercise A's card make the active exercise unambiguous.

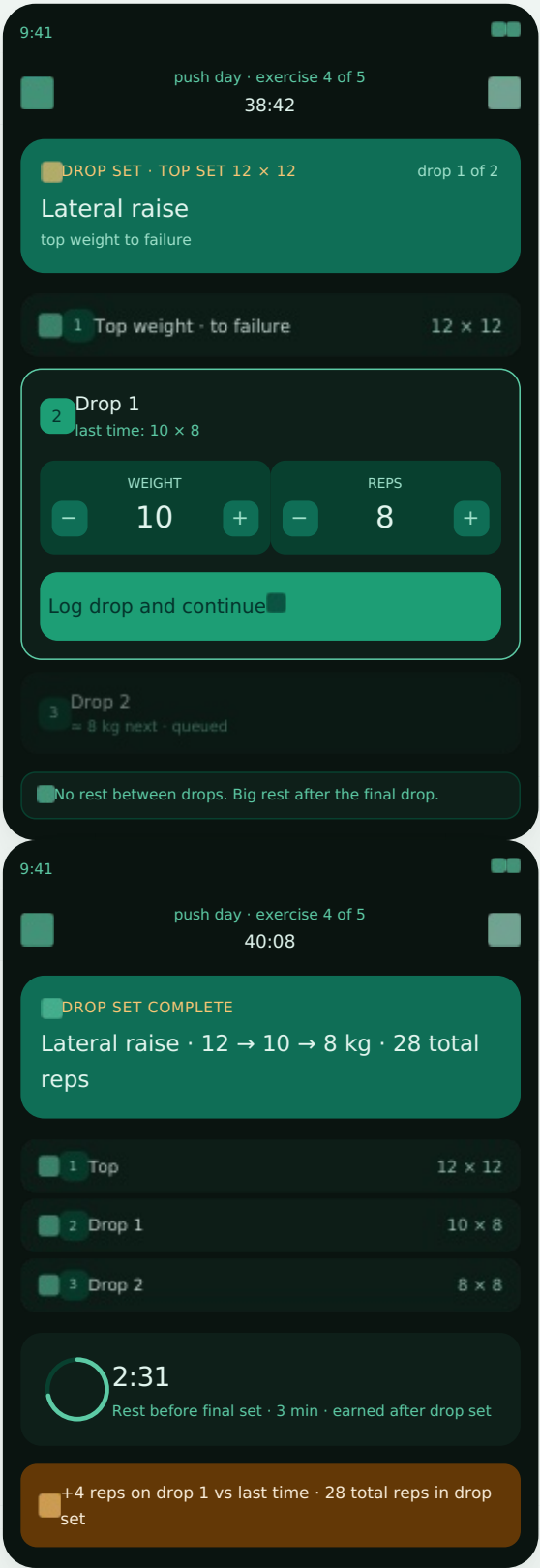
The button label "Log and move to B" is more informative than "Next" — the user knows exactly what tapping does. After A is logged, the card flips: A becomes the muted state, B becomes active, and the button reads "Log and finish round." Rest only fires after the *round* — never between A and B.

DATA AND BEHAVIOURAL IMPLICATIONS

A superset is a `set_group` row with `kind = superset`. Each exercise's sets within the group carry `set_group_id` and `set_group_position` (0 for A, 1 for B). The rest timer is stored on the `set_group` (not the individual sets) and the active workout store knows to suppress between

linked sets.

Screen 14 — Drop set logger



ROLE

Drop sets rendered as one extended set with multiple drops. Two frames: mid-drop-set (active drop 1 of 2) and complete (all drops logged, resting).

RATIONALE

The trending-down icon for drop sets is a deliberate semantic choice — it's *intentional* descent, not failure. The hero card's "top set 12 × 12" framing makes clear what the top weight was.

The data model unifies supersets and drop sets under the `set_group` pattern. Both are linked sequences with one rest at the end. The "+4 reps on drop 1 vs last time" pill is the drop-set-specific PR recognition — even though the user didn't lift heavier, they did *more work* at the same drop weight, and that's worth celebrating.

DATA AND BEHAVIOURAL IMPLICATIONS

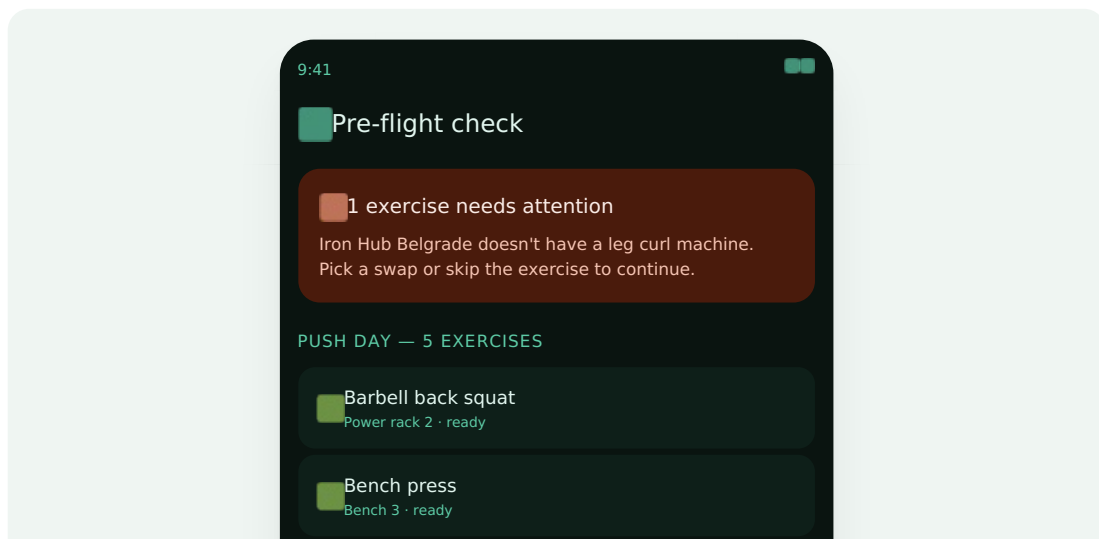
A drop set is a `set_group` with `kind = drop_set`. Position 0 is the top set, positions 1, 2, 3... are the drops. The rest after the final drop is encoded on the group with `rest_seconds = 180` (3 min default) rather than the standard 2 min. PR detection on drop sets has two paths: traditional PR on the top set, and "drop PR" on each drop (improvement at the same drop weight versus the last drop set of this exercise).

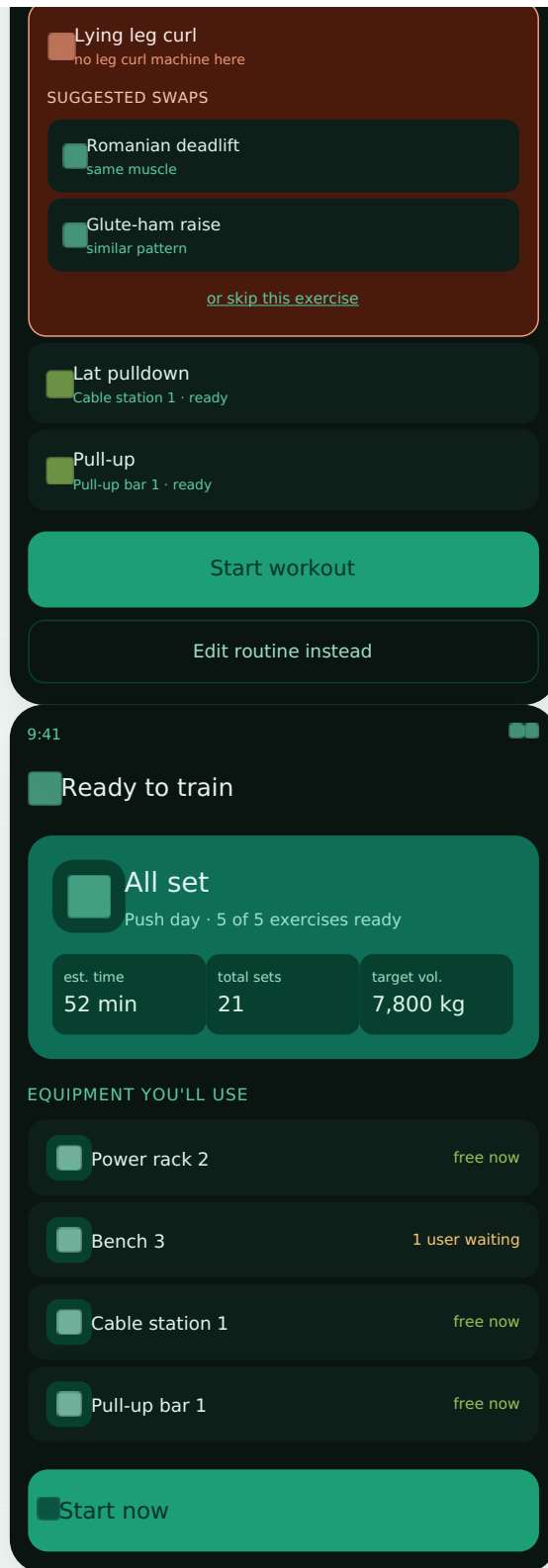
SECTION

Logging surface completion

Three screens that round out the logging surface — the pre-flight check, the QR scan flow, and the rest states.

Screen 16 — Routine pre-flight check





ROLE

The moment the user taps "Start workout" on their planned routine, the app checks whether the current gym has every piece of equipment that routine requires. If anything is missing, this screen appears before the workout starts. The user can swap, skip, or proceed.

RATIONALE

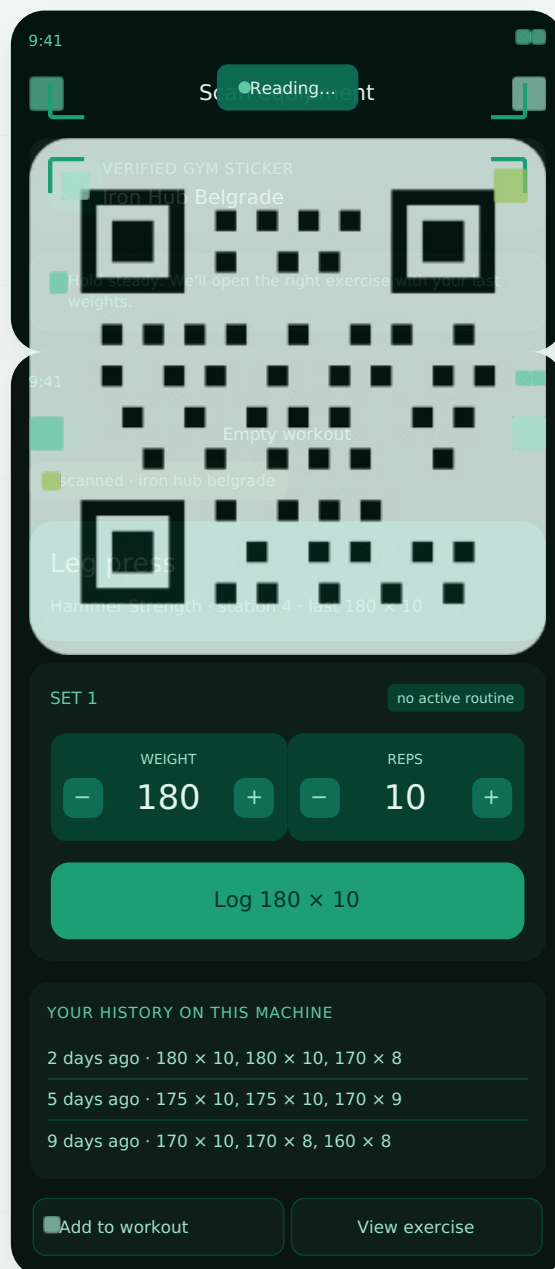
The coral semantic palette for the warning is restrained — alert without alarm. The single-sentence description tells the user exactly what's wrong. The two suggested swaps are ranked: "same muscle" first (the safer programmatic substitute), "similar pattern" second.

The primary button stays enabled because the workout can still start once a swap is selected — defaulting to the top suggestion on tap. The realtime "free now / 1 user waiting" indicators presume the gym operator tier has equipment-usage tracking enabled; if unavailable, the indicators silently disappear.

DATA AND BEHAVIOURAL IMPLICATIONS

Pre-flight runs at "Start workout" tap time. The app loads the routine's exercises, looks up each one's required equipment category, then checks `gym_equipment` for matching items at the current gym. Suggested swaps come from an `exercise_substitutions` table that maps each exercise to alternates within the same muscle group, ordered by similarity score.

Screen 17 — QR scan to log



ROLE

The user is in front of a machine, opens the app's QR shortcut, scans the sticker. Within ~300ms they're on the smart logger with exercise + gym + equipment instance all pre-bound and last-session values pre-filled.

RATIONALE

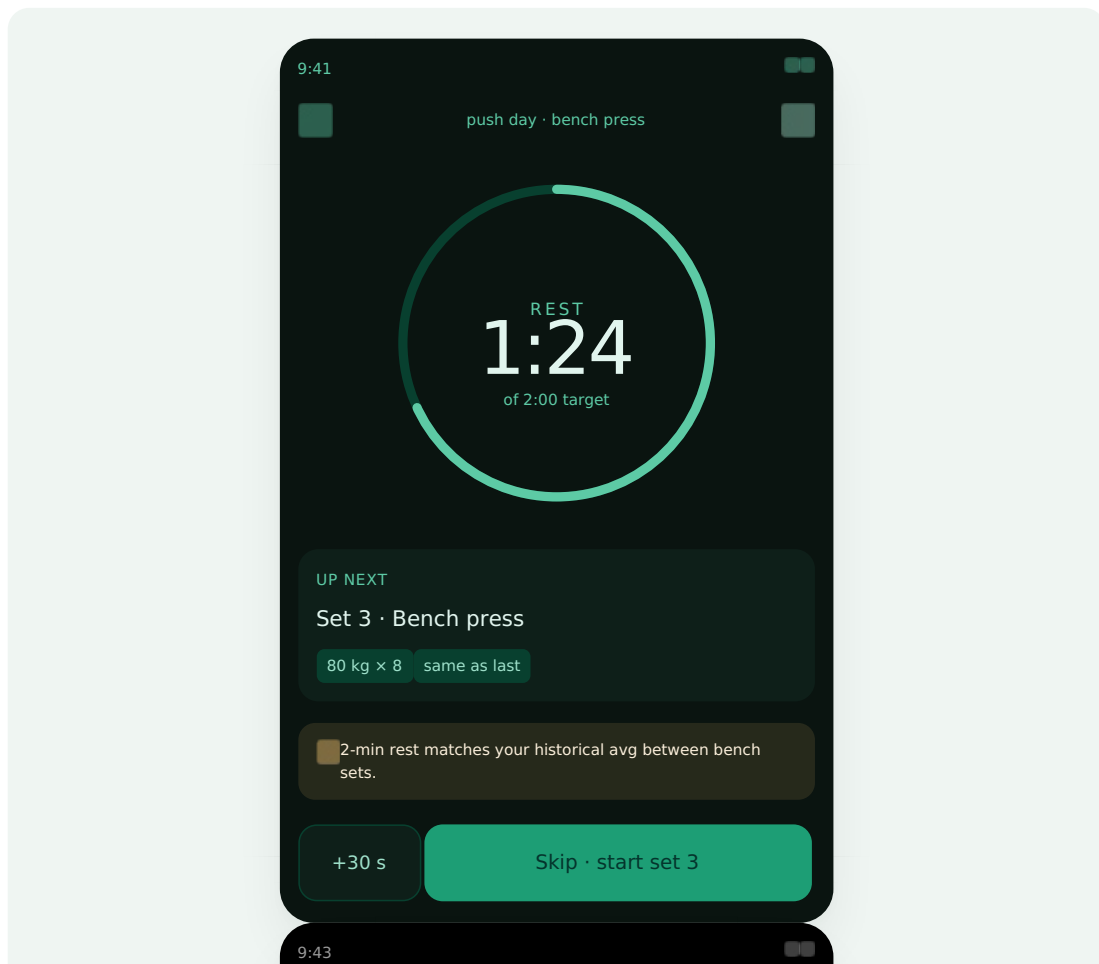
The corner-bracket targeting overlay in the saturated accent green ties the scan moment to the brand. The "Reading..." toast is small but reduces user anxiety — they're not sure if anything is happening. The shield-check icon on the gym card is the security model made visible. Counterfeit QR codes silently fail to validate and never reach the landing screen.

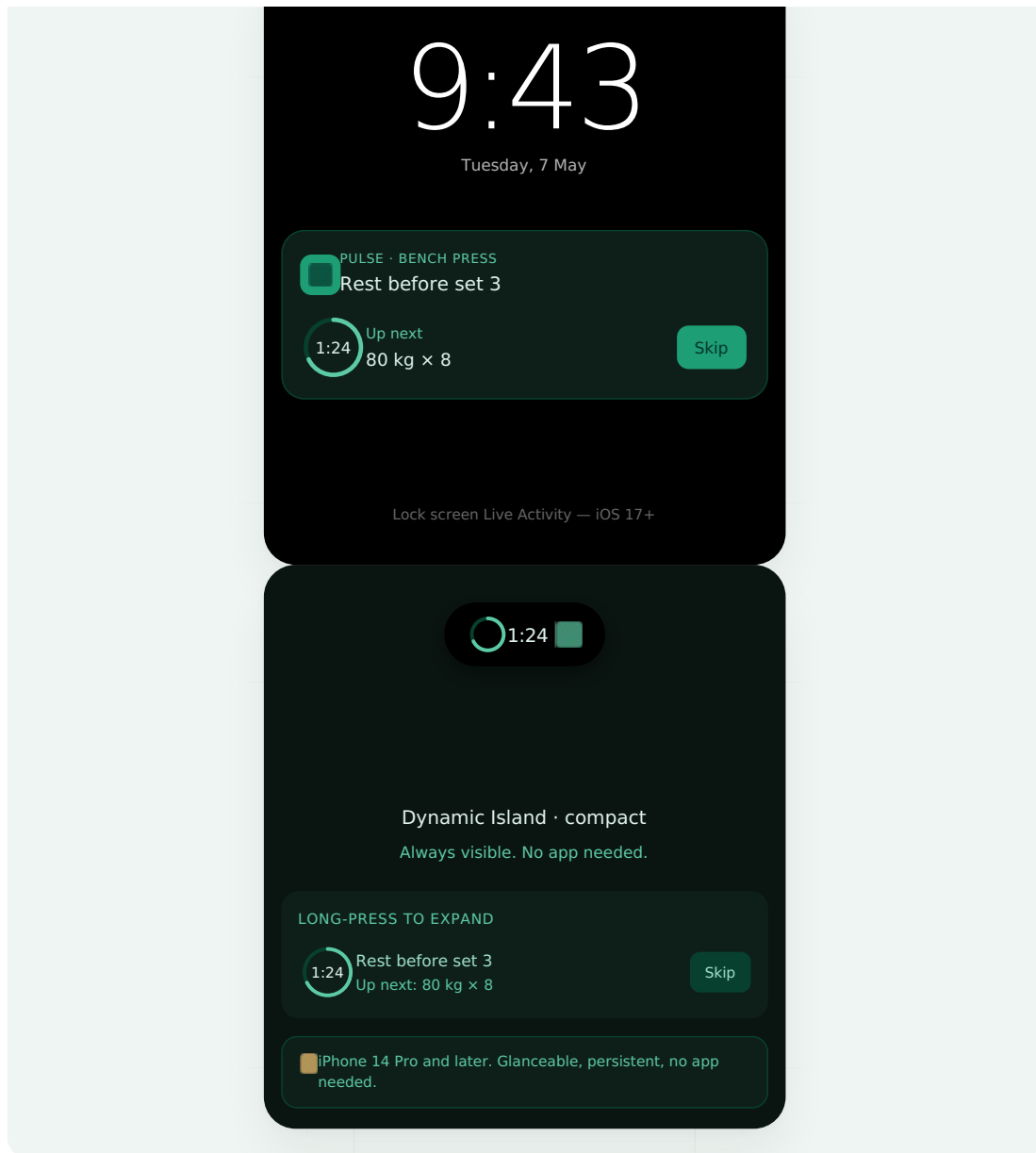
The "history on this machine" card is the QR-flow's killer feature. Because we bound `gym_equipment_instance_id` to each set, we can answer "what did I do on Hammer Strength station 4 specifically?" instantly.

DATA AND BEHAVIOURAL IMPLICATIONS

The QR sticker encodes a signed JWT with `gym_id`, `equipment_instance_id`, `exercise_id`, and an expiration timestamp. The JWT is signed by Pulse's server with a rotating key. On scan, the app verifies the signature locally using the cached JWKS from `/api/jwks` (rotated daily, cached for 7 days). The session metadata includes `entry_source = qr_scan` for analytics.

Screen 18 — Rest state, three views





ROLE

The rest timer rendered three ways: in-app full screen, iOS Live Activity on the lock screen, and Dynamic Island compact view. All three read the same data from the active workout store.

RATIONALE

The 220×220px progress ring is large enough to read across a gym floor. The wall-clock-anchored timer means it stays accurate even if the app is backgrounded or the phone is locked mid-rest. The "2-min rest matches your historical avg" insight is the Pulse copy principle — explain the why, not just the what.

The Live Activity is the killer iOS-specific feature. Users with iPhone 14 Pro and later see the Dynamic Island constantly. A glanceable rest timer in the Island is a category-defining differentiator that Hevy doesn't ship.

DATA AND BEHAVIOURAL IMPLICATIONS

The rest timer's `started_at` timestamp is persisted to the workout session state in MMKV. The remaining time is computed as `target_seconds - (now - started_at)` rather than decremented in a JS interval. This means the timer survives app backgrounding, phone lock, and incoming phone calls.

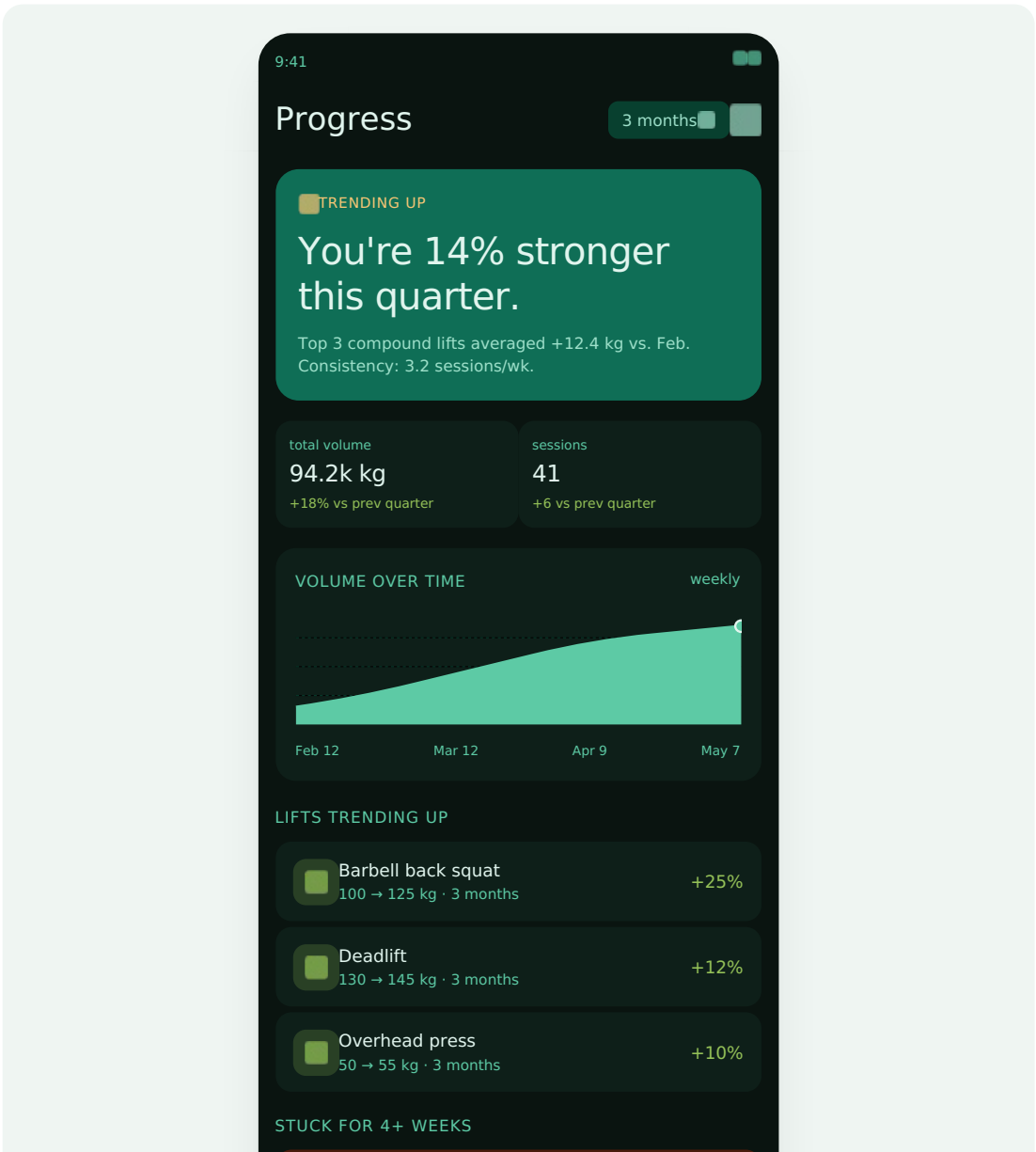
The Live Activity is updated via `expo-live-activity`'s `update()` method. On iOS, the activity is created when the user taps "Log set" (starting the rest), updated every 5 seconds, and dismissed when the user taps "Start next set" or the timer expires.

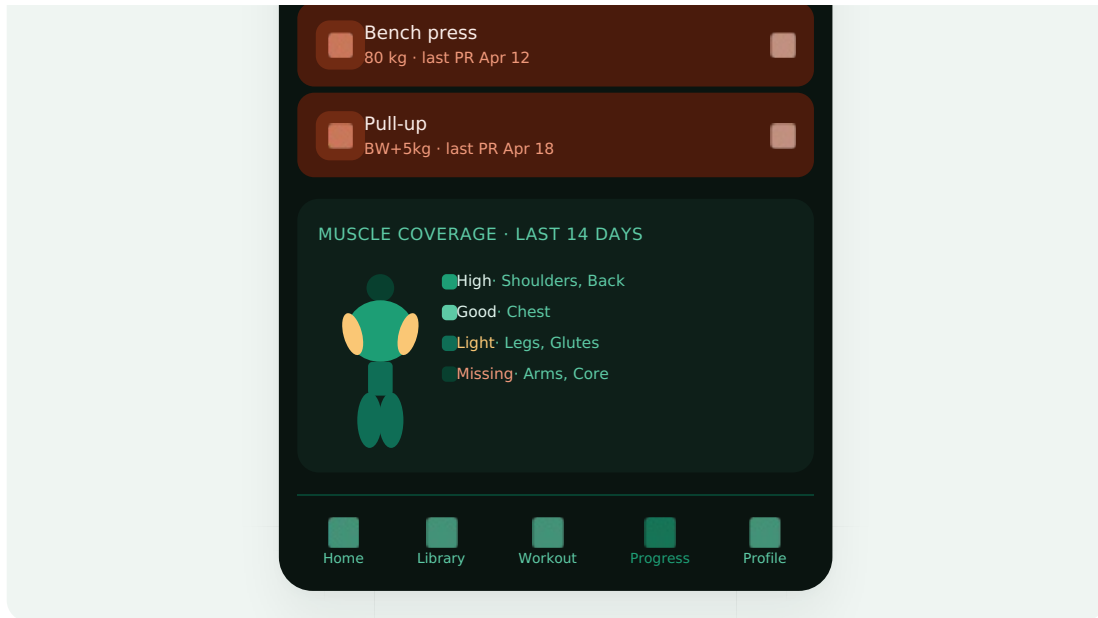
BATCH 2

Progress and analytics

Six screens covering the analytical surface of the app. The job here is *insight*, not statistics — every chart answers a specific question a lifter asks themselves, in plain language.

Screen 19 — Progress dashboard





ROLE

The progress tab landing surface. Surfaces the answer to "how am I doing right now?" as quick answers, not raw metrics. Top-line: am I trending up or down. Below: which lifts are moving and which are stuck. At the bottom: a muscle-group heatmap.

RATIONALE

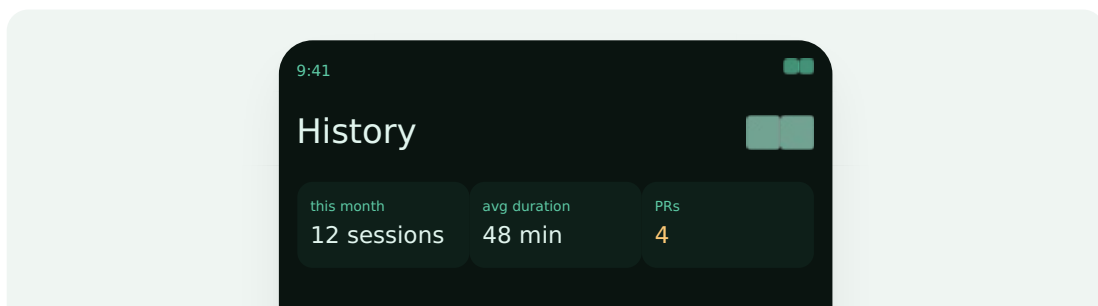
The dashboard leads with a *headline* — "You're 14% stronger this quarter" — written like a sportscaster's read of the last three months. This is the strongest emotional moment in the entire analytics surface. The supporting numbers explain the headline in plain English.

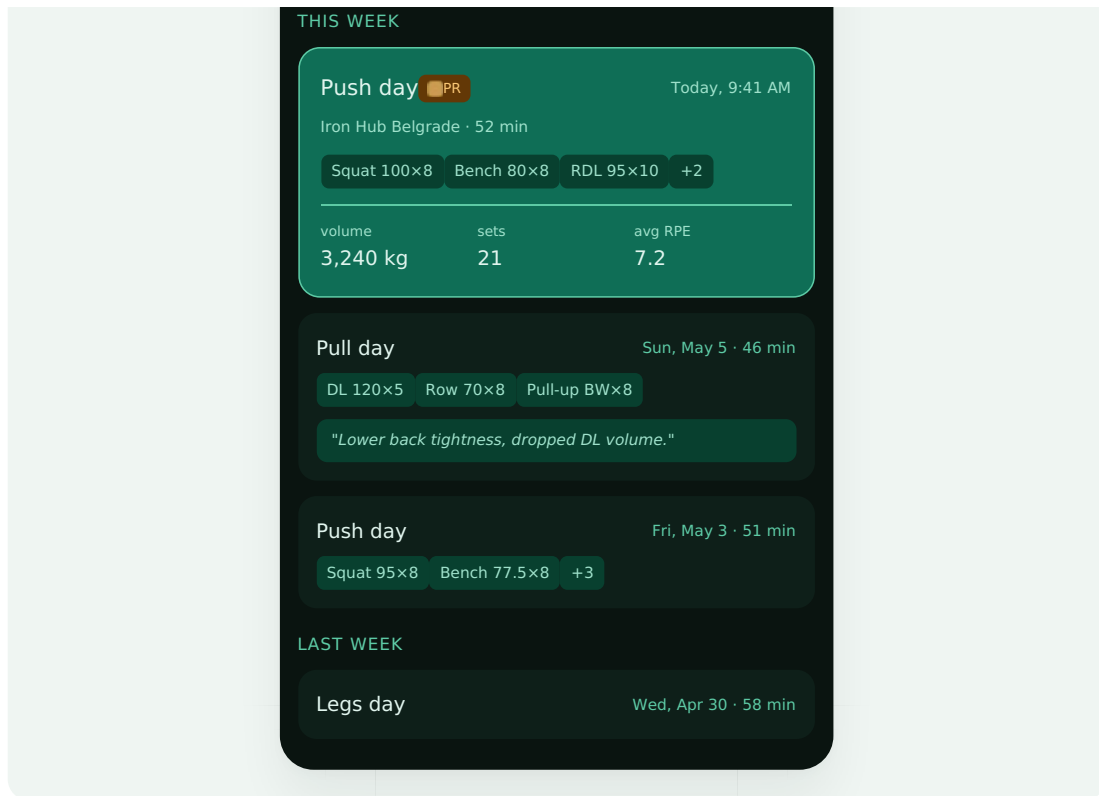
"Lifts trending up" and "Stuck for 4+ weeks" are the *actionable* parts. Stuck lifts use the coral palette to mark them as concerns. The muscle coverage heatmap is the Hevy-killing differentiator — a 30-second intervention into the user's training program.

DATA AND BEHAVIOURAL IMPLICATIONS

The "14% stronger" computation: estimated 1RM averaged across the user's top 3 most-trained compound lifts. Threshold for the "trending up" framing: $\geq 5\%$ improvement. Stuck lifts: any exercise with no PR in the last 28 days AND with at least one session in that window. The muscle coverage heatmap aggregates by `exercise.primary_muscle` over the last 14 days, bucketing into four tiers based on the user's historical baseline.

Screen 20 — History feed





ROLE

The user taps "history" or scrolls past today's training. The journal view — every workout in reverse chronological order, with enough glanceable info per session that the user remembers what they did without tapping in.

RATIONALE

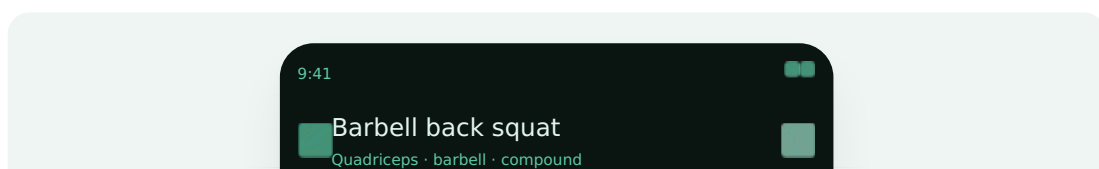
The most recent workout gets the hero treatment because that's what the user wants to verify. The PR badge top-right is the celebratory marker. Each card's "+N more" exercise chip summarizes overflow so the user gets the gist without scrolling each card.

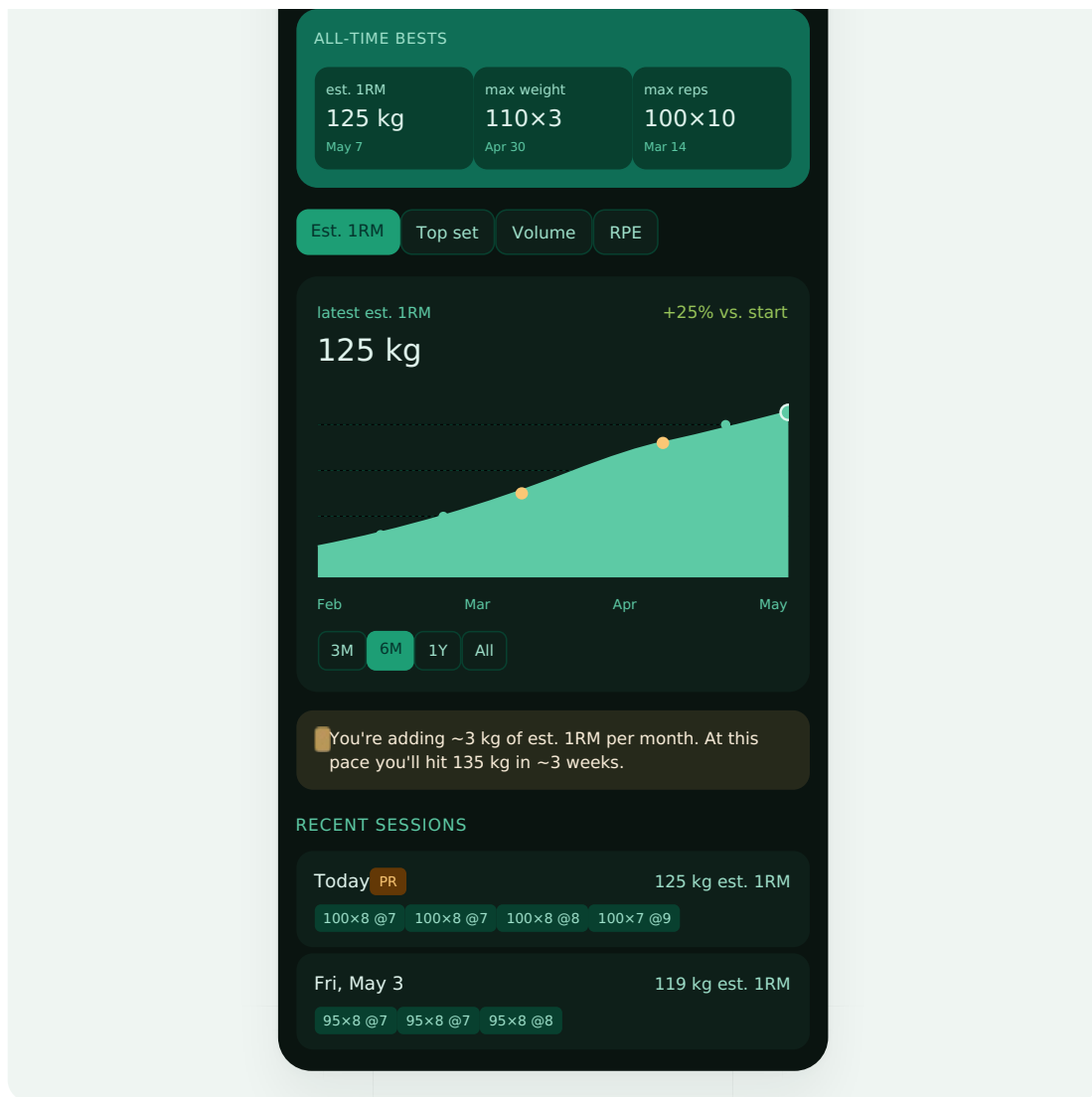
Notes are surface-rendered in history feed (not buried behind a tap) because notes are the feature that turns history into a journal. Users genuinely write things like "left knee felt off" or "tried wider grip" and want to find those notes later.

DATA AND BEHAVIOURAL IMPLICATIONS

The hero session is the most recent `workout_session`. The exercise chips show distinct exercises with their top set. Computing "top set" per exercise involves picking the heaviest weight × reps combination, prioritising weight for compound exercises and total reps for isolation exercises.

Screen 21 — Exercise progress chart





ROLE

The single-exercise deep dive. The page a serious lifter visits regularly. Multiple metric toggles, range selector, every set logged as a dot you can scrub.

RATIONALE

The all-time-bests trio is what serious lifters check first — "did I beat any of my three numbers today?" Showing 1RM, max weight, and max reps separately because each is a meaningful achievement type.

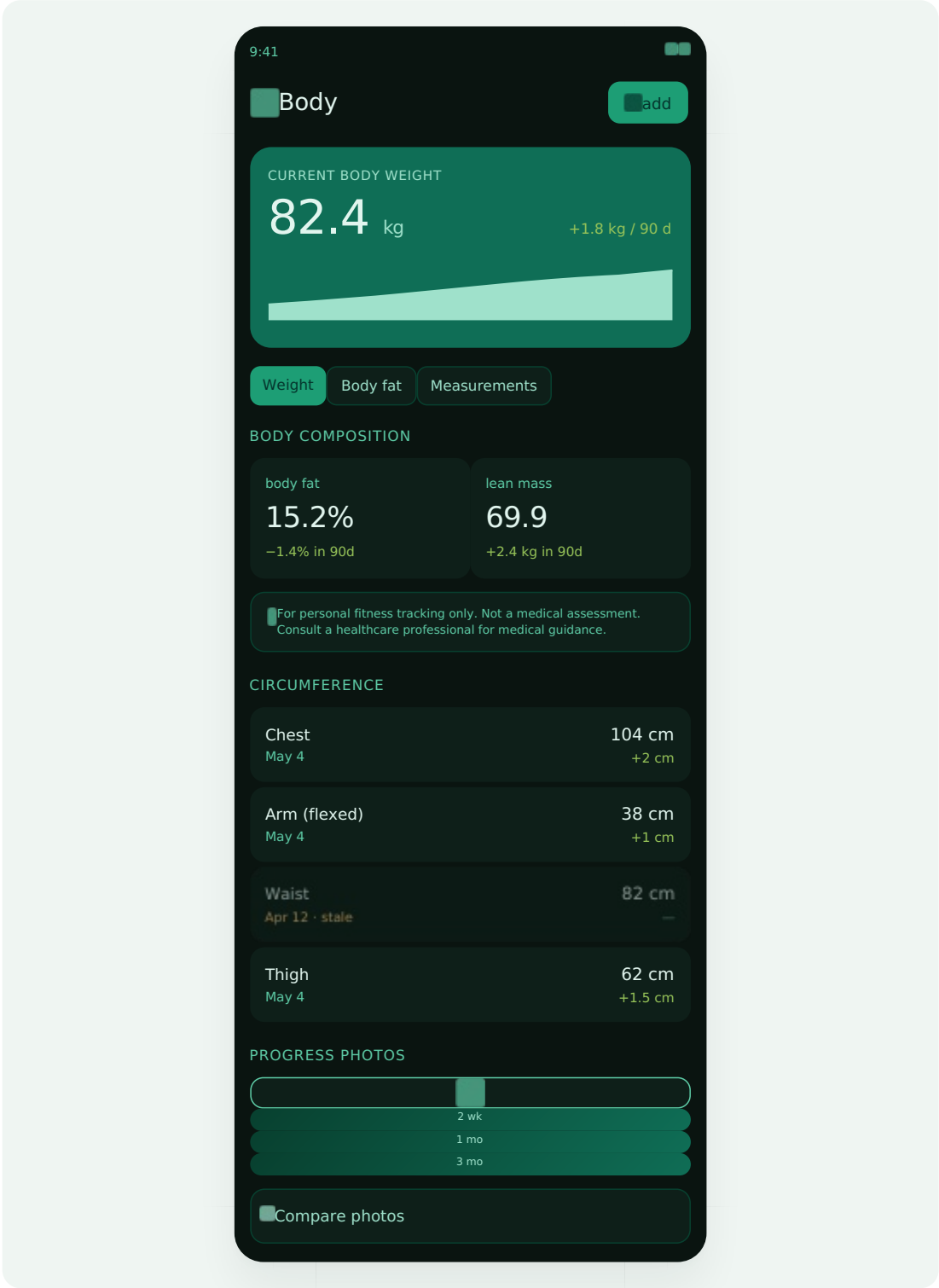
The four metric toggles let the user reframe the chart. The default (estimated 1RM) is right because it unifies weight and reps into a single comparable number across all set styles. PR sessions are amber-dotted on the chart so the user can visually scan their history and spot the breakthrough moments.

The insight pill below the chart is the differentiating feature — derived projection in plain language, anchored in the user's actual data: "You're adding ~3 kg per month. At this pace you'll hit 135 kg in ~3 weeks." This is the moment the app proves it's worth premium.

DATA AND BEHAVIOURAL IMPLICATIONS

The four metric calculations: Est. 1RM uses Epley formula, Top set picks max weight × reps, Volume sums across all sets per session, Avg RPE averages `rpe_inferred`. The projection is a linear regression over the last 8 sessions' estimated 1RMs. If $R^2 < 0.5$ (noisy data), the projection is suppressed because we don't want to make false promises.

Screen 22 — Body measurements



ROLE

The user logs weight, body composition, and circumference measurements here. Pulse takes the regulatory positioning seriously — every body-comp number carries the "not a medical assessment" disclaimer.

RATIONALE

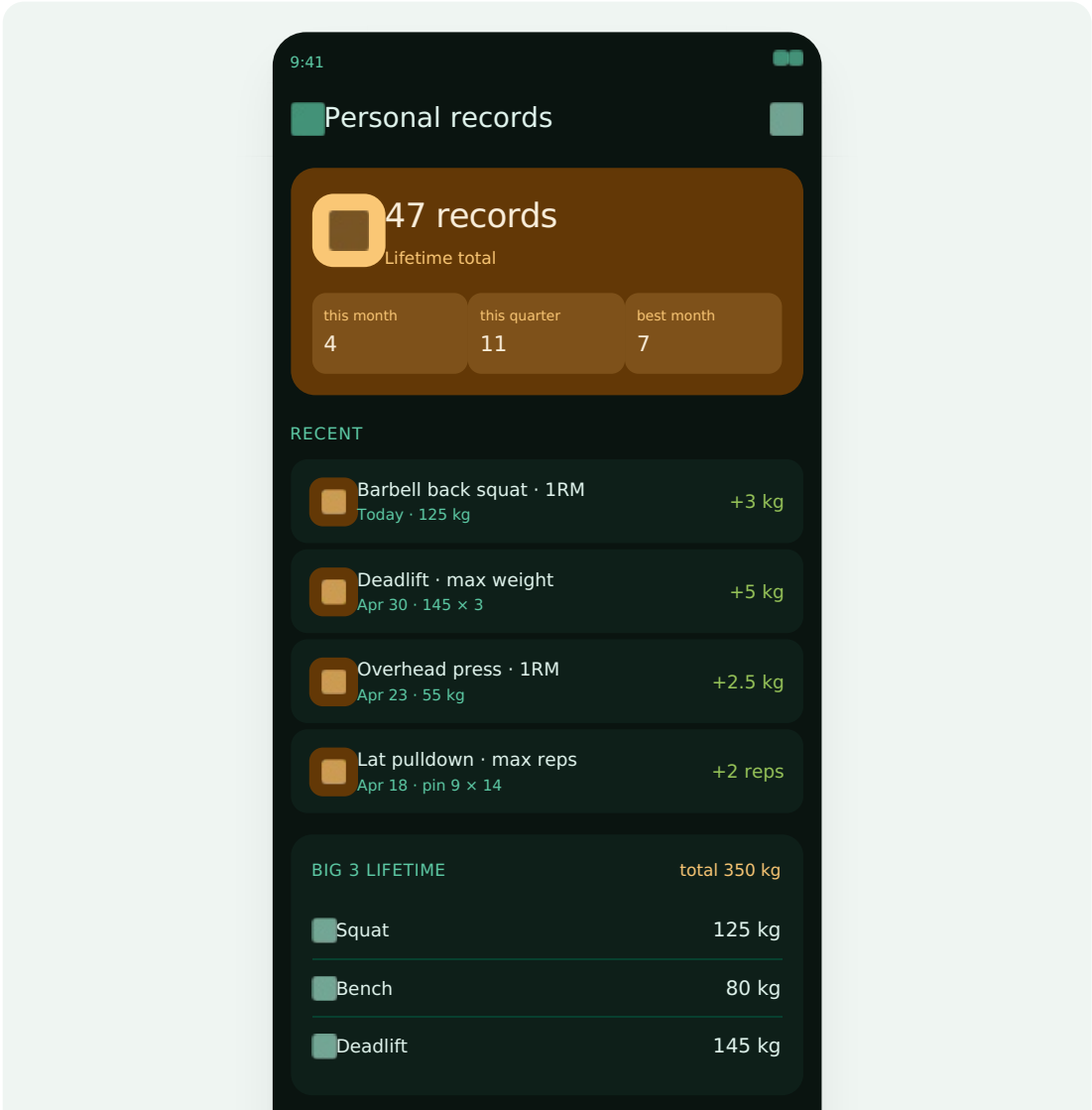
Body weight gets the hero treatment because it's the daily-logged metric. The +1.8 kg in 90 days is rendered in muted green because the *trend* matters, not the absolute number — the user could be in a strength phase (gaining intentionally) or cutting (losing intentionally).

The disclaimer card is **mandatory** and shown wherever body composition appears. This is the regulatory pattern from the fitness-domain-expert agent: every body-comp surface gets the disclaimer to defend against an App Review "is this a medical app?" challenge. Stale measurements (last logged >14 days for circumference) get a soft visual nudge.

DATA AND BEHAVIOURAL IMPLICATIONS

Body composition entries are user-input rows in `body_measurements` with `recorded_at`, `weight_kg`, `body_fat_percent`, `lean_mass_kg`, and per-site circumference values. Progress photos are stored in Supabase Storage at `users/{user_id}/body_photos/{photo_id}.jpg` with row references in `body_photos`. The 90-day delta calculation finds the closest measurement to "exactly 90 days ago."

Screen 23 — Personal records



All records by exercise

ROLE

Every PR the user has ever set, organised by exercise, sorted by recency. The user taps "View all" from home's milestones or the trophy icon from anywhere.

RATIONALE

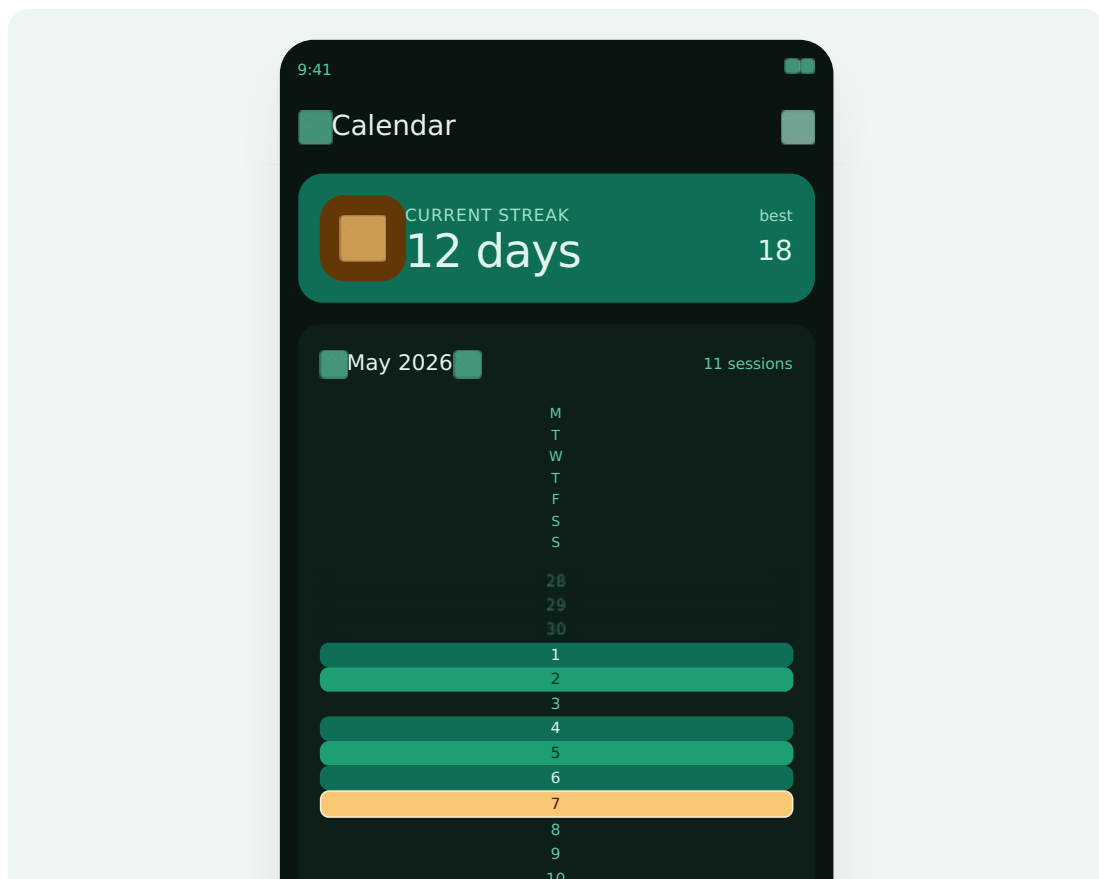
The PR screen leans into the amber palette deliberately — these are the celebratory moments. The lifetime-total card shows 47 records with subtimings — this-month, this-quarter, best-month. The "best month" stat is small but motivating — it tells the user what they're capable of when they're really dialed in.

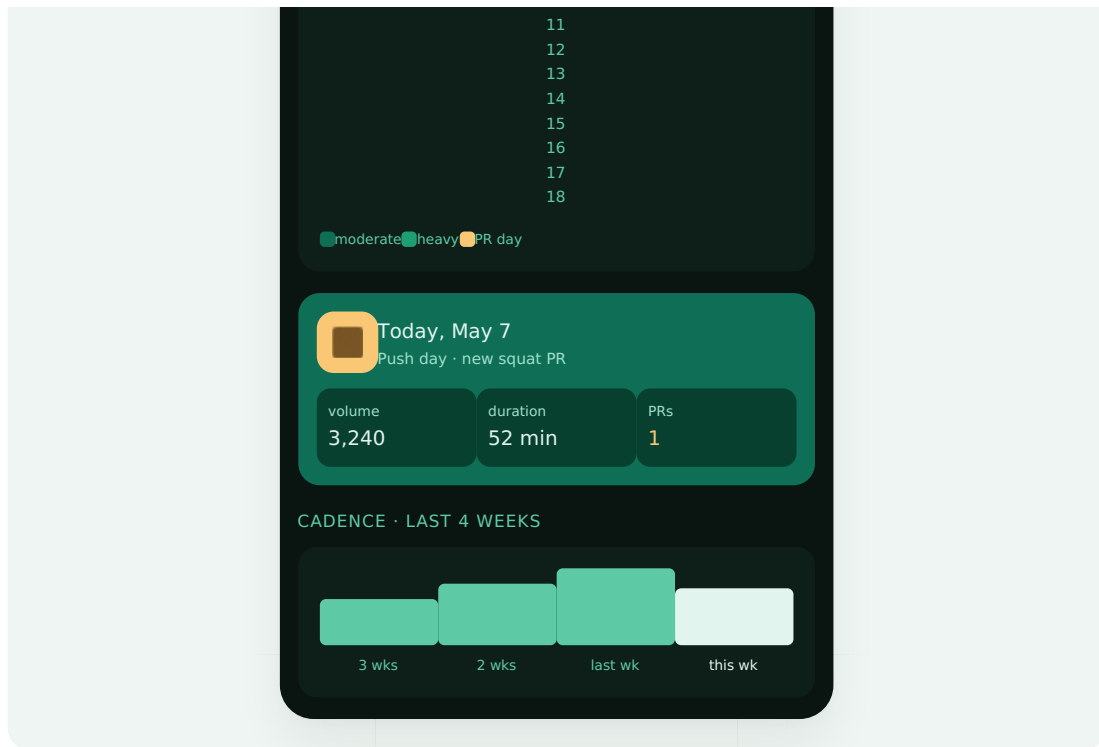
Each PR row shows the delta ("+3 kg") because that's how much they improved, more meaningful than just the new record. The Big 3 lifetime card aggregates squat/bench/deadlift into a powerlifting total — the canonical metric for strength athletes.

DATA AND BEHAVIOURAL IMPLICATIONS

The `personal_records` table is populated by a server-side trigger on `workout_sets` insert. The three PR types tracked per exercise: estimated 1RM, max weight for any reps, max reps at any weight. Each gets its own row in `personal_records` with a `record_type` enum.

Screen 24 — Calendar and streak





ROLE

The time view of training. Streak counter, monthly heatmap, selected-day card, weekly cadence chart.

RATIONALE

The streak counts training days, not consecutive days. If the user trains Monday/Wednesday/Friday, the streak ticks up each training day rather than breaking on rest days. This matches how lifters actually think about consistency.

The calendar's three-tier color coding (moderate / heavy / PR-day) is calibrated to the user's recent average — "heavy" means heavy *for you*, not in absolute terms. The cadence chart at the bottom shows the last 4 weeks of session counts. The user can see at a glance if this week's pace is on track or behind. Soft motivator, not a guilt-trip.

DATA AND BEHAVIOURAL IMPLICATIONS

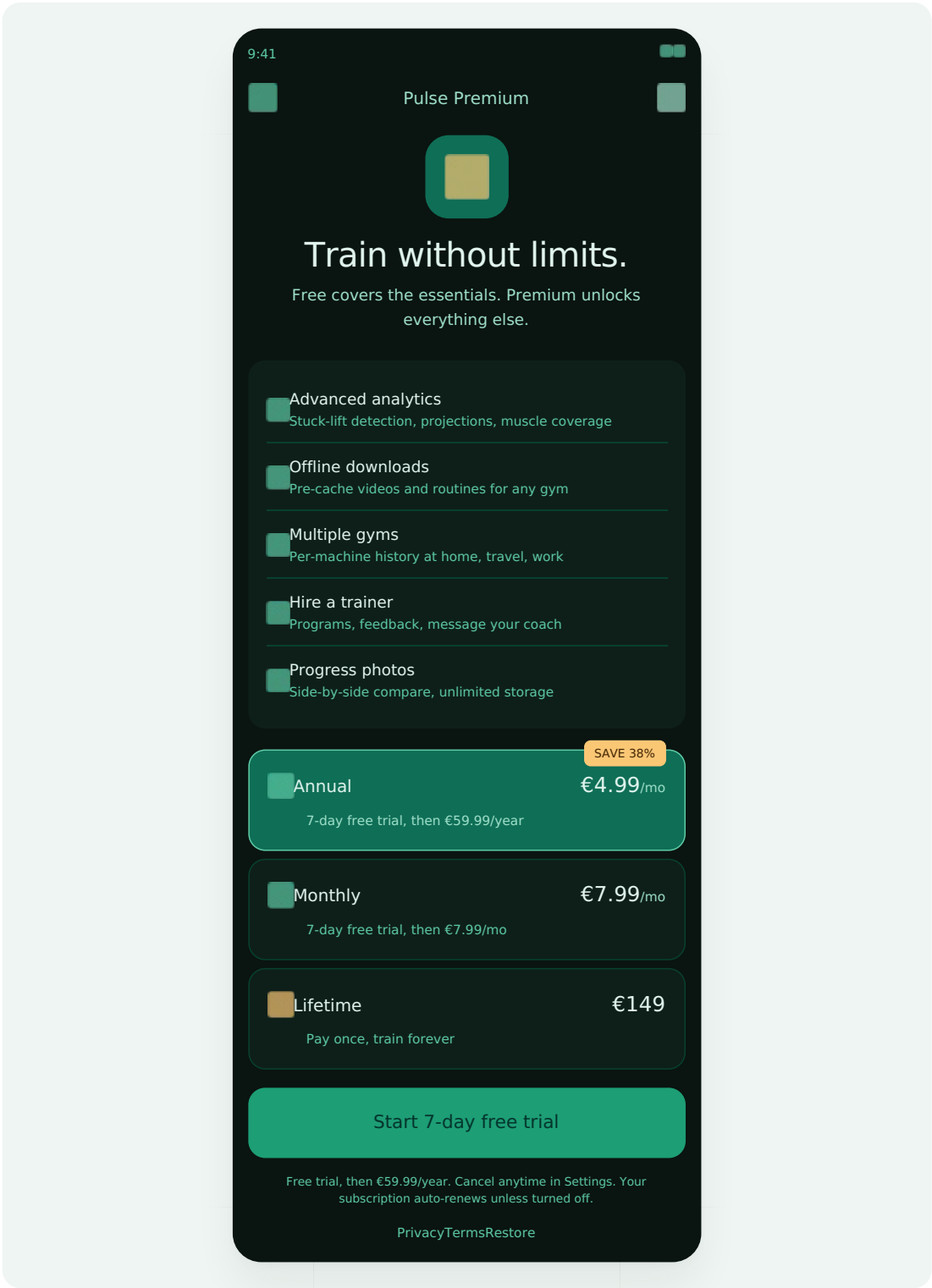
The streak query: starting from today, walk backward day-by-day and count consecutive *training* days. Calendar tiering: per-day total volume bucketed against the user's 30-day rolling average. Heavy = $> \text{average} + 1 \text{ std dev}$. Moderate = within $\pm 1 \text{ std dev}$. PR day = any day with a `personal_record` row.

BATCH 3

Power features

Five screens covering the systems that make the app feel premium and trustworthy — subscriptions, notifications, settings, offline downloads, and integrations.

Screen 25 — Paywall



ROLE

The moment a user hits a premium feature. Free up to a meaningful threshold (logging, history, basic routines, one gym); premium for advanced analytics, video coaching, multiple gyms, offline downloads, trainer features.

RATIONALE

The headline followed by an honest framing — "Free covers the essentials. Premium unlocks

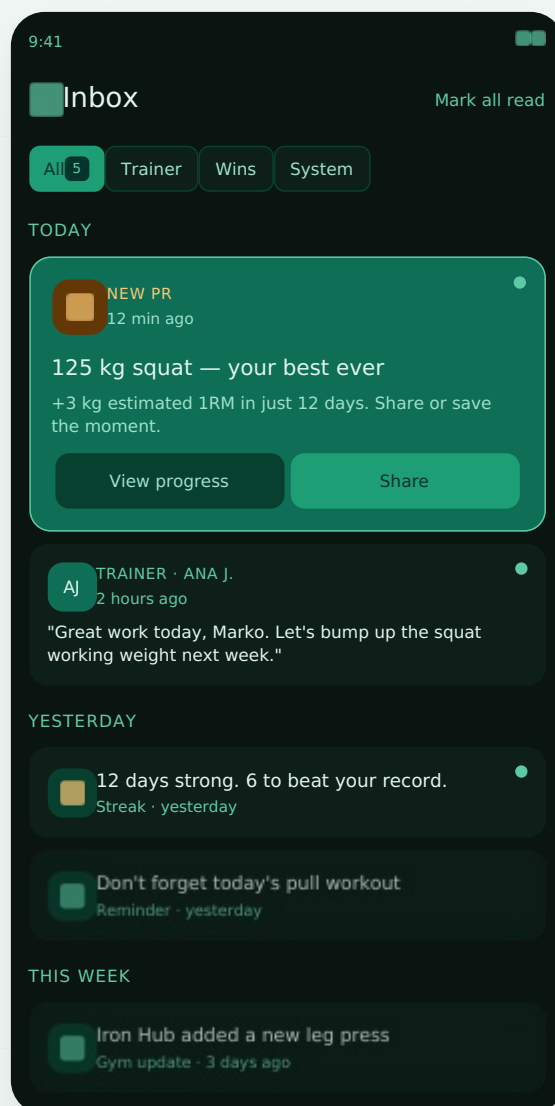
everything else" — sets the right relationship. Each of the five benefits has a specific concrete promise tied to features we've built, not generic marketing.

Three tiers because lifetime is high-margin and reduces churn anxiety for committed users; monthly is for the cautious; annual converts at the highest rate. The 7-day free trial is required for Apple App Store and aligns with the conversion data from the deep stack research (5-9 day trials convert ~37% to paid versus ~25% for shorter trials).

DATA AND BEHAVIOURAL IMPLICATIONS

Subscriptions are managed by RevenueCat from day one (~1% MTR fee after \$2.5K). Entitlements are mirrored server-side via RevenueCat webhooks to a `user_subscriptions` table for fast access without API calls. The premium features check this status locally — no network call required to gate features.

Screen 26 — Notifications inbox



ROLE

Push notifications, in-app messages, trainer messages, and PR/streak achievements all surface here. Grouped by category so it's not a flat firehose.

RATIONALE

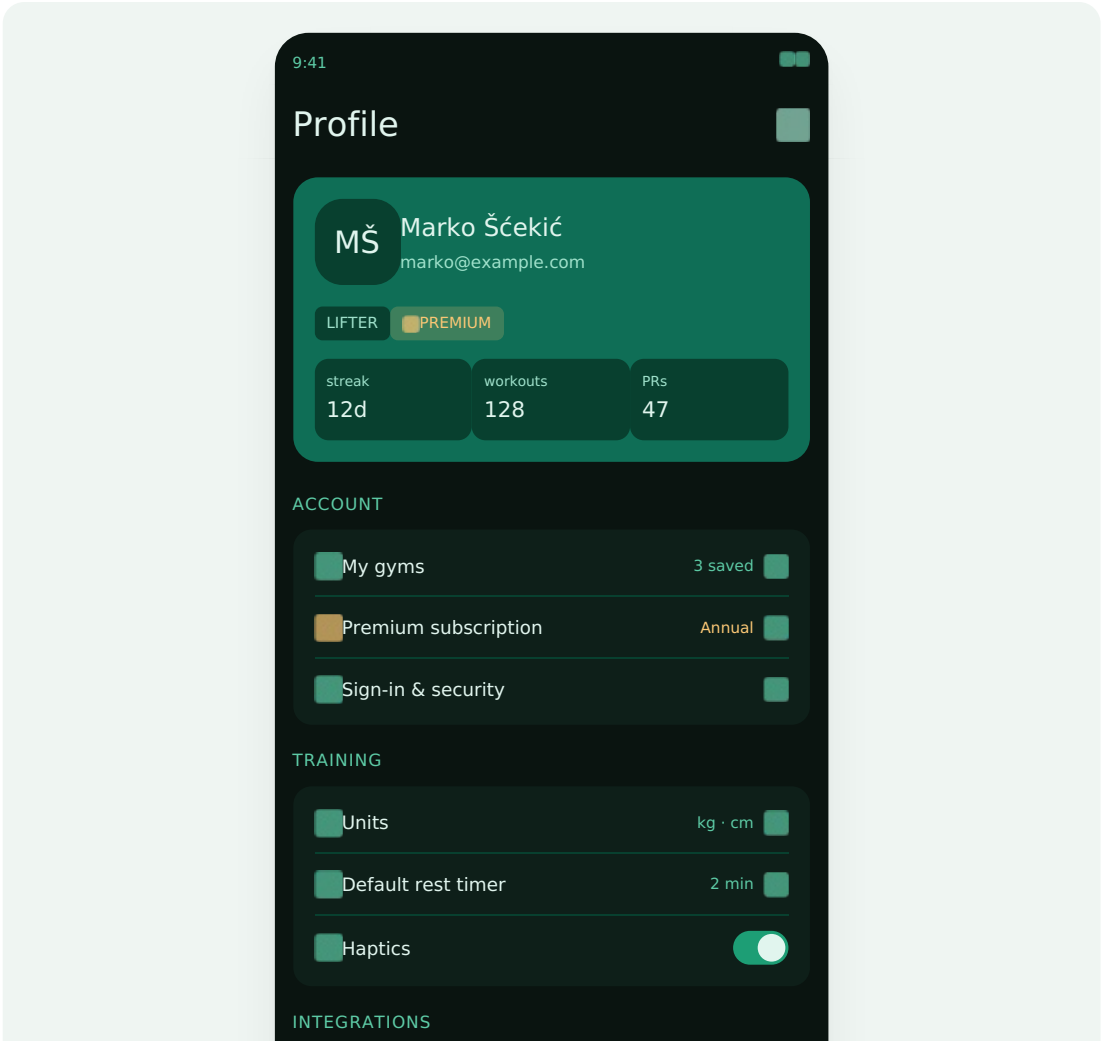
Categories at the top let the user filter to the noise level they want. Counts on tabs (the "5" badge) show what's unread. The PR notification gets the hero card treatment because it's a celebratory moment. Inline actions ("View progress" + "Share") let the user act without leaving the inbox.

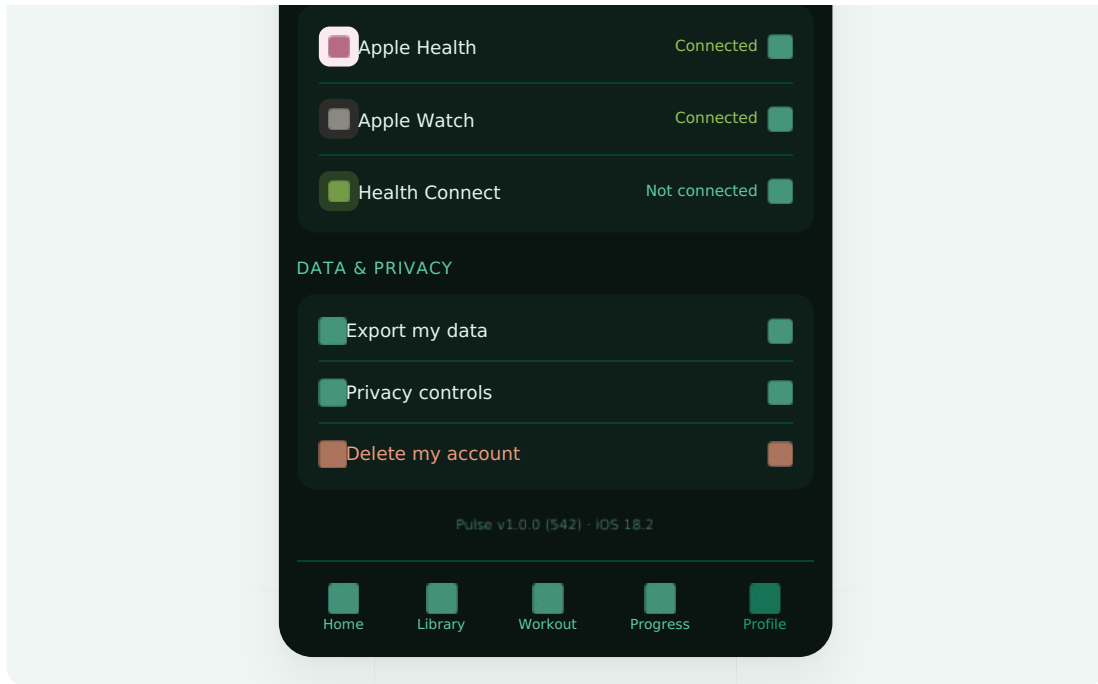
Streak notifications are anticipatory ("6 to beat your record") rather than guilt-trippy. Read notifications fade to 75% opacity so unread items dominate visually.

DATA AND BEHAVIOURAL IMPLICATIONS

Notifications come from multiple sources: push (Expo Notifications), in-app messages (Supabase Realtime), and derived events (PRs detected by server triggers, streak milestones computed nightly). All notification records live in a `notifications` table with `kind`, `actor_id`, `subject_id`, `payload` JSONB, and `read_at` timestamp.

Screen 27 — Settings and profile





ROLE

The profile tab. Everything the user can configure, organised by concept (account, training preferences, devices, support, privacy). The hero is the user's identity block.

RATIONALE

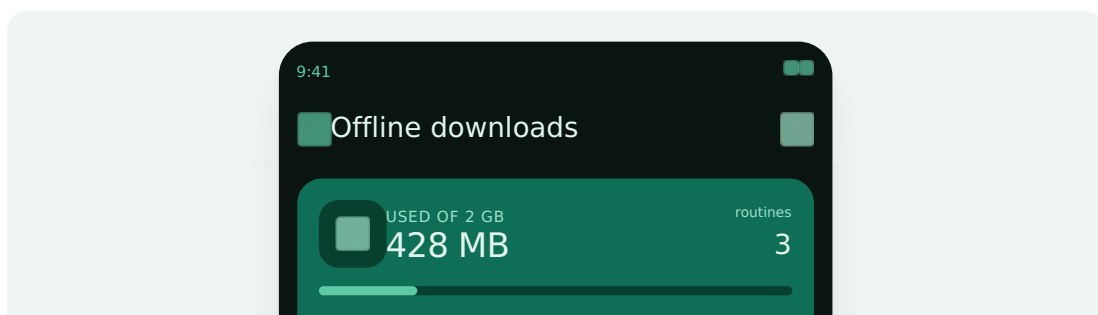
The identity card tells the user three things they care about: who they are, what kind of user, and what they've achieved. The whole card is tappable to open account detail.

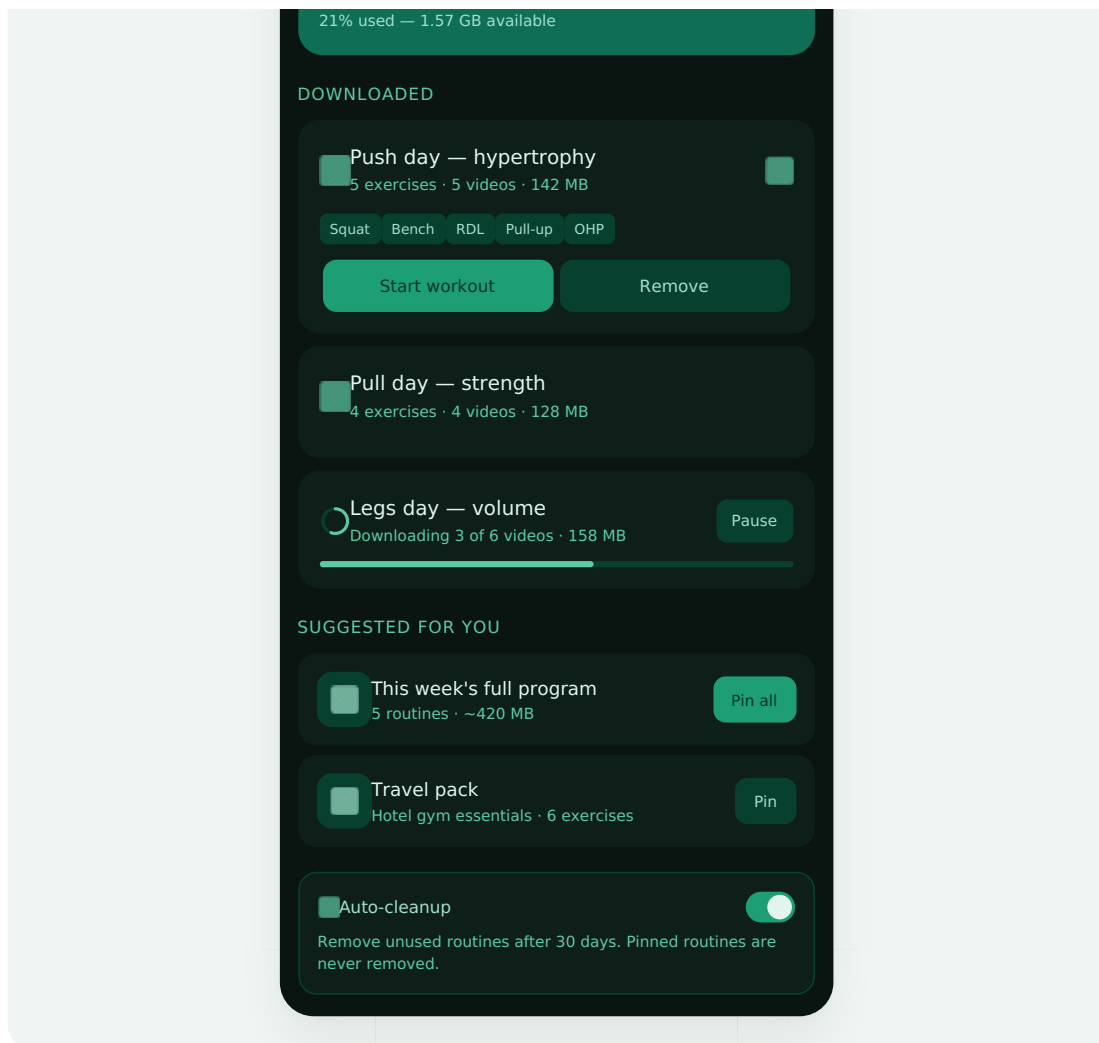
The haptics toggle is a real toggle (not a chevron) because it's a binary the user flips often. The integration rows show platform-branded icons. The "Delete my account" row is rendered in coral so it stands apart from regular actions, but it's still in the same list — it's a normal user right under GDPR, not an emergency.

DATA AND BEHAVIOURAL IMPLICATIONS

Account settings are stored in `user_preferences` with sensible defaults. Local SQLite is the source of truth; the server is a sync mirror. Data export generates a JSON archive via a background worker. The user receives an email with a signed download link valid for 24 hours. Account deletion is a hard delete after a 30-day grace period (GDPR Article 17 compliance).

Screen 28 — Offline routine download





ROLE

Premium feature. The user pre-caches routines (videos, exercise instructions, reference images) to disk for unreliable-signal training environments.

RATIONALE

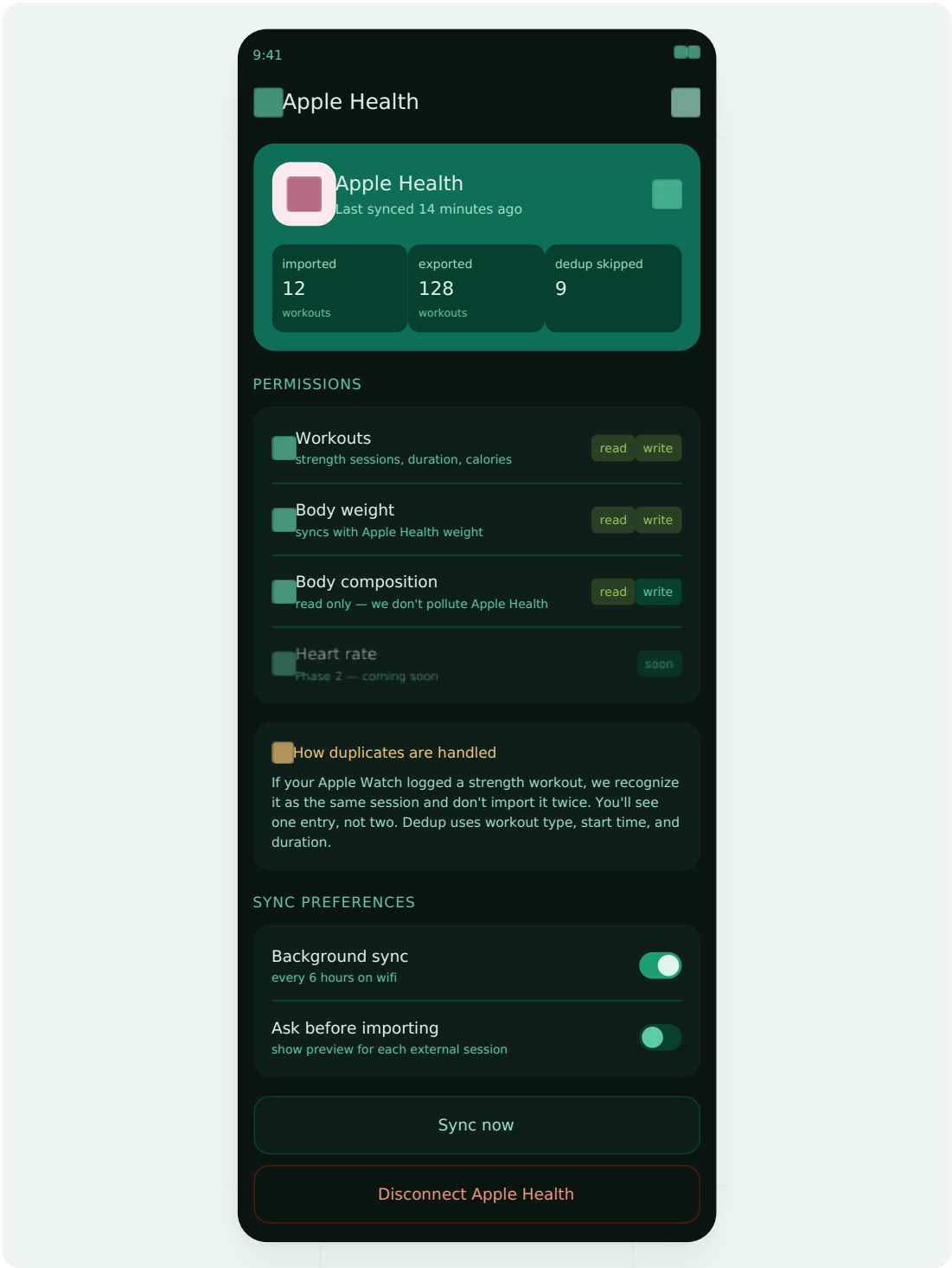
The wifi-off in the status bar sells the offline use case immediately. The storage cap (2GB default) maps to the eviction policy we encoded in the agent files. "Suggested for you" is anticipatory — the app proposes pinning the week's program or a Travel pack based on calendar integration hints.

The auto-cleanup toggle gives the user control without being threatening — unused routines auto-clear after 30 days, pinned routines never.

DATA AND BEHAVIOURAL IMPLICATIONS

Routine downloads pre-fetch Mux video files via `expo-video-cache` to the iOS Cache directory, with cache keys based on routine + video version. The cache supports HLS streaming offline. Total cache size is checked before download; the user is prompted if free space is insufficient. Auto-cleanup runs nightly on cold launch and removes the least-recently-played non-pinned routines until the cache is under 80% of the cap.

Screen 29 — Apple Health integration detail



ROLE

The user taps "Apple Health" from settings. This screen shows the granular per-data-type permissions, what data flows in vs. out, and the deduplication explanation.

RATIONALE

The hero card's three stats prove dedup is working without making the user audit anything. Per-data-type pills are scoped, not blanket — this is the App Review-mandated pattern for HealthKit in 2026.

We deliberately do not write body composition to Apple Health because the user's source-of-truth there is usually a smart scale, and we don't want to pollute it. The dedup explanation is what App Reviewers read to confirm we're not creating duplicate health entries. The disconnect button in coral is the GDPR Article 7 user right — disconnect anytime.

DATA AND BEHAVIOURAL IMPLICATIONS

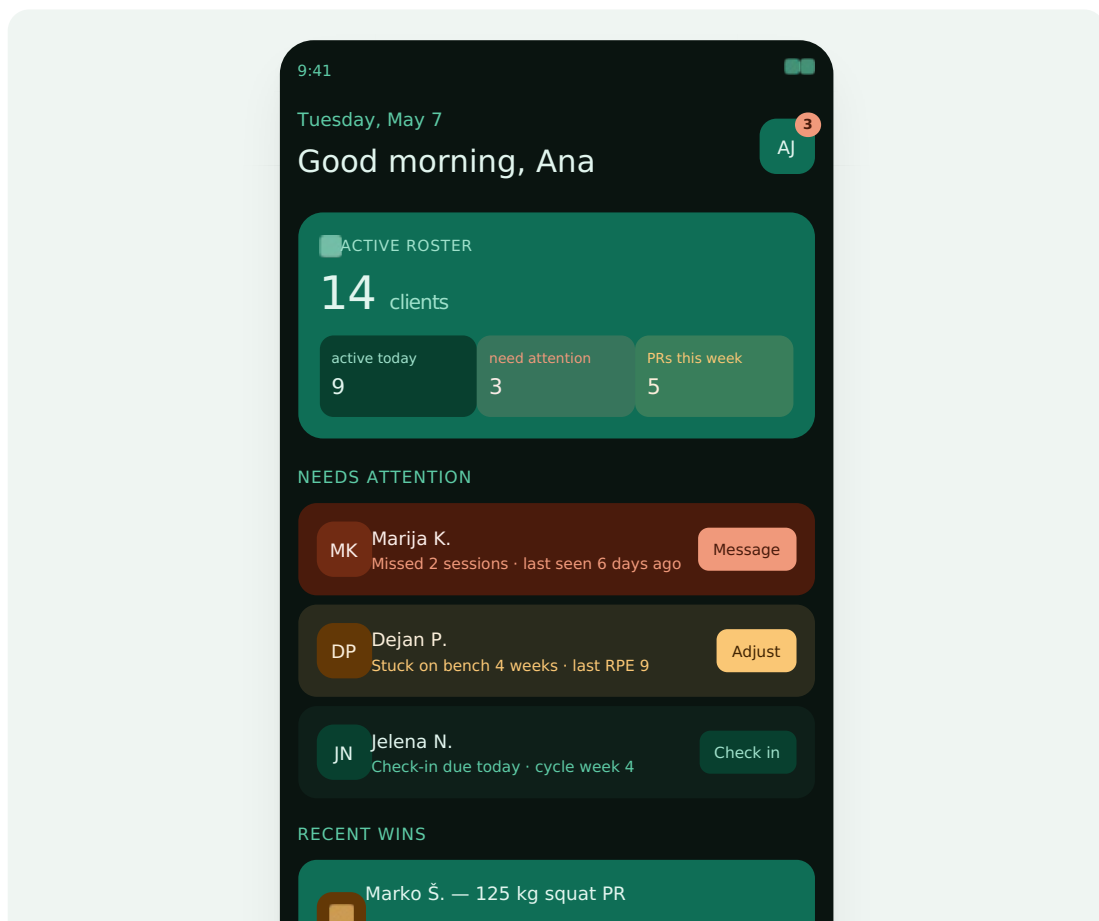
HealthKit integration uses the `expo-health` library. Permission requests are per-data-type rather than blanket-all, which matches Apple's preferred pattern. Dedup logic: on each Watch-logged workout import, we check for an existing local workout with `workout_type = strength` and `start_time` within ± 60 seconds and `duration` within $\pm 20\%$. If found, skip the import and increment the `duplicates_skipped` counter.

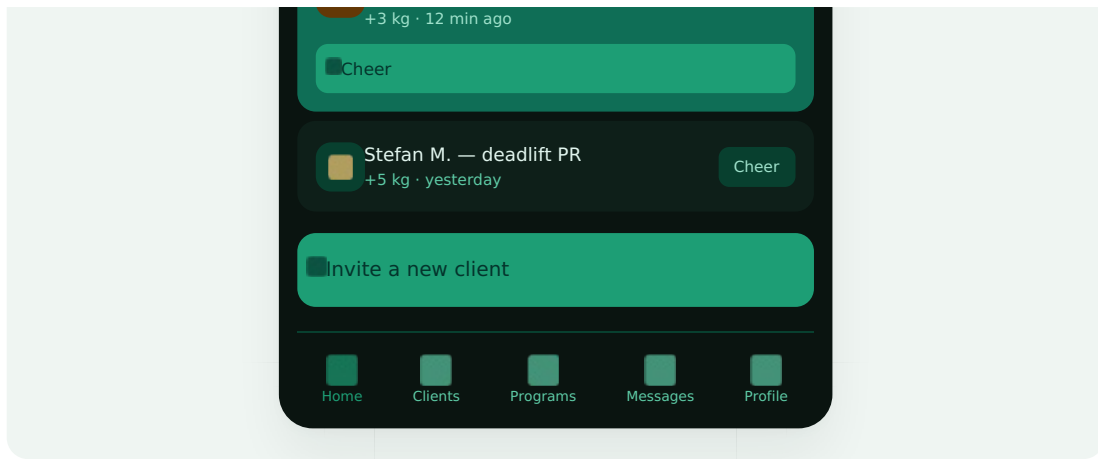
BATCH 4

Multi-tenant surfaces

Six screens for the trainer and gym operator user types. Same backend, same design language, role-specific information density and metric sets.

Screen 30 — Trainer dashboard





ROLE

When a personal trainer signs in, the home tab is replaced with this. The trainer's mental model is "who do I need to think about right now?" — not "what should I train?"

RATIONALE

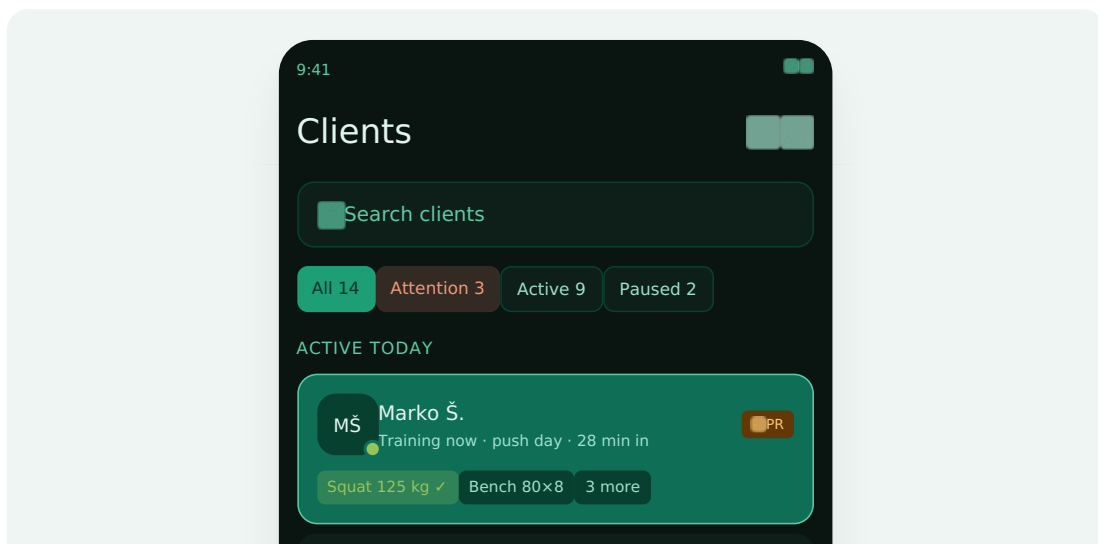
The trainer has a **different bottom tab bar** — Workout becomes Clients, Library becomes Programs. They're managing other people's training, not their own.

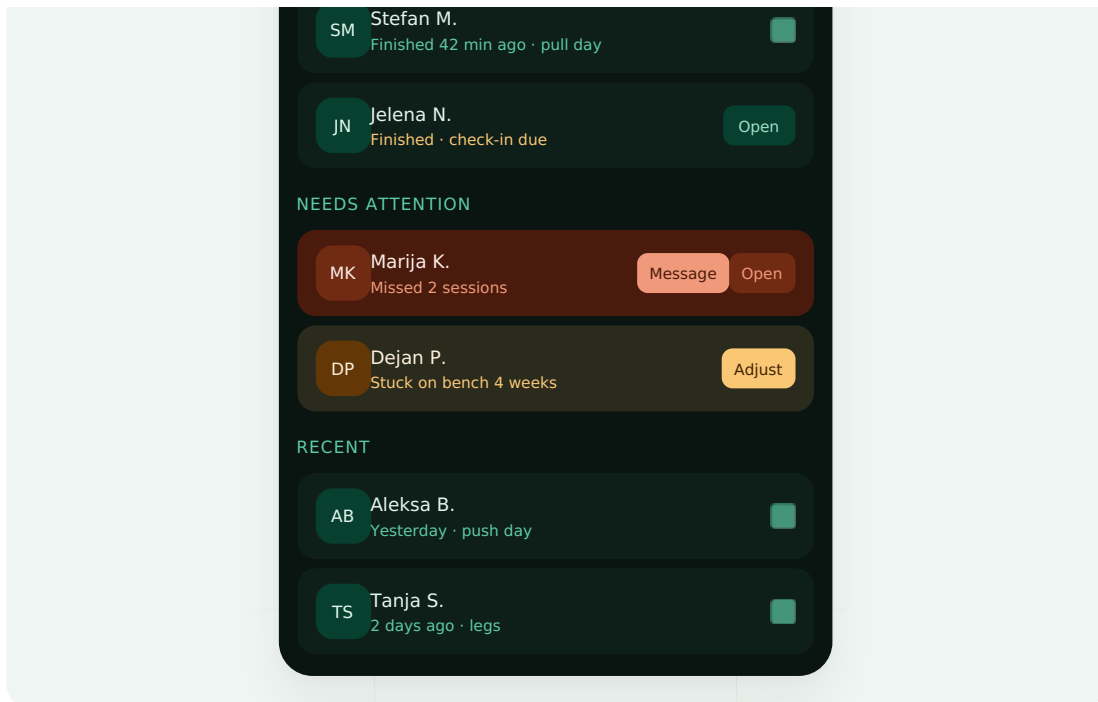
"Need attention" uses the coral palette — actionable column. The trainer's eye is drawn here first. Each client row uses progressive coral intensity matching urgency. "Cheer" is the one-tap celebration pattern — sends a pre-composed congratulatory message via the in-app messaging surface, removing the friction of typing.

DATA AND BEHAVIOURAL IMPLICATIONS

Trainer-client relationship is via Clerk Organizations: the trainer is the org owner, clients are org members. PowerSync's `trainer_clients` sync stream filters by Clerk org membership. Attention detection runs nightly. "Recent wins" subscribes to `personal_records` filtered by clients-of-this-trainer, ordered by `created_at` descending.

Screen 31 — Client list





ROLE

The trainer taps "Clients." Full roster with sort/filter, search, and at-a-glance status per client.

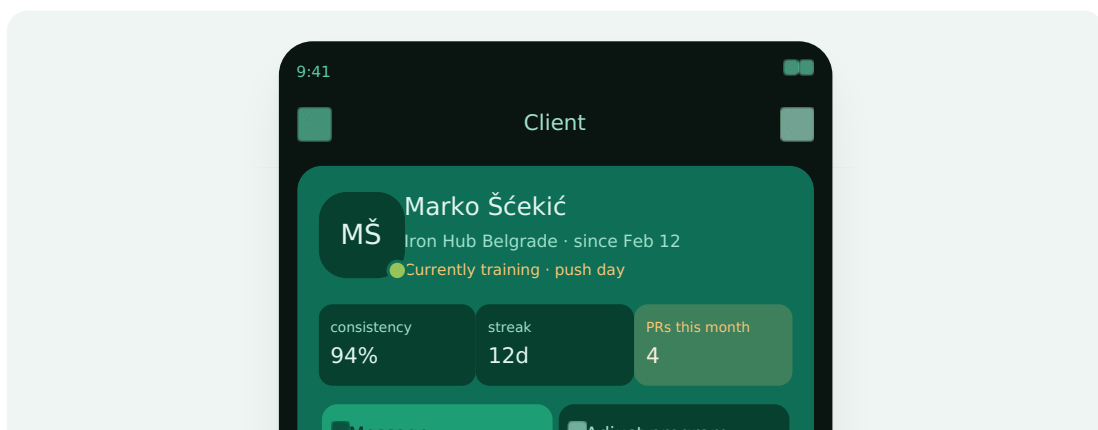
RATIONALE

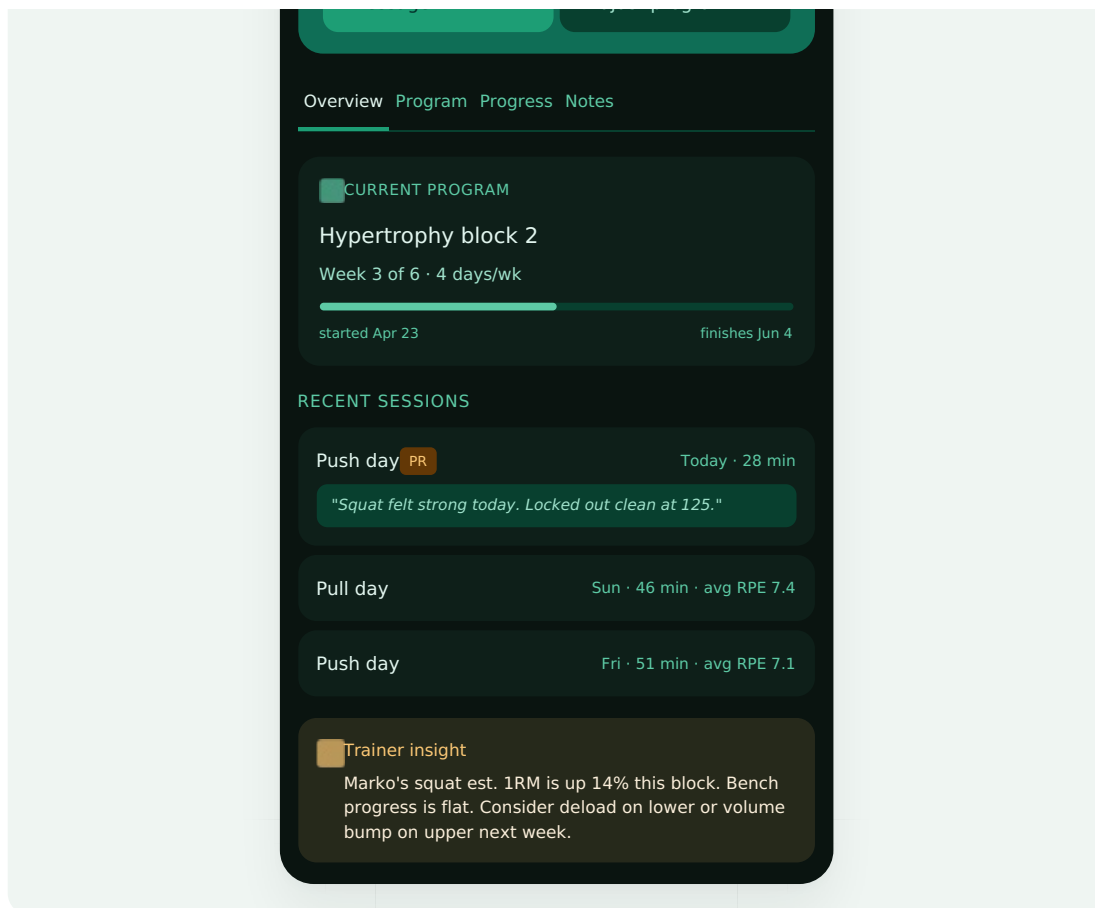
The currently-training card is the trainer's superpower — live training visible in real-time. The "Training now" subtitle + green presence dot signals this is realtime data. Quick-action buttons let the trainer act without drilling in. "Message" opens the chat; "Open" goes to client detail.

DATA AND BEHAVIOURAL IMPLICATIONS

The "currently training" status comes from `workout_sessions` rows with `started_at` but no `ended_at`, scoped to clients of this trainer via the `trainer_clients` sync stream. Live training updates use Supabase Realtime subscriptions — when a client logs a set, the trainer's roster updates within seconds without polling.

Screen 32 — Client detail





ROLE

The trainer's view of one client. Client profile, current program, message thread, shortcuts for the actions trainers actually do — adjust the program, send a note, mark a check-in done.

RATIONALE

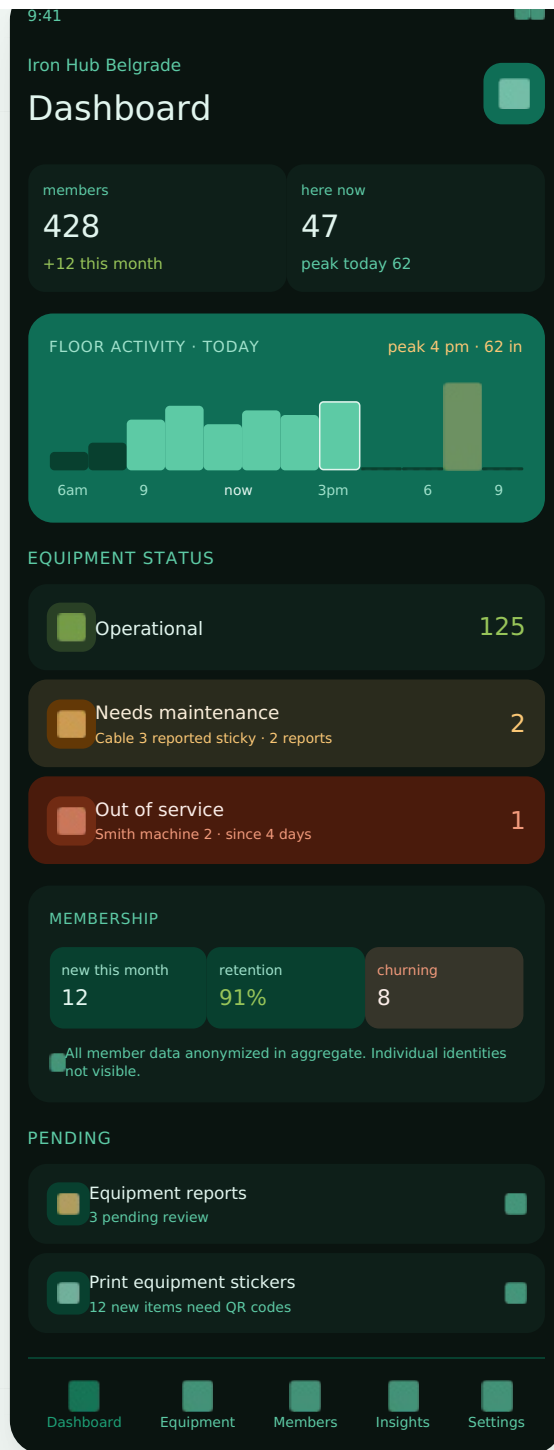
The hero card extends the lifter's identity card with trainer-relevant stats. The amber PR tile uses the celebratory palette — wins are celebrated on both sides. The two primary actions ("Message" + "Adjust program") are the two things trainers actually do. Everything else lives in the inner tabs.

The "Trainer insight" is the AI-derived programming recommendation surfaced specifically to the trainer. Same data source as the lifter's projection insight, framed for the role.

DATA AND BEHAVIOURAL IMPLICATIONS

Trainer-to-client messaging is stored in `messages` with `thread_id` + `sender_id` + `recipient_id`. The "Adjust program" action opens the routine builder in trainer-edit mode, with the client's active program pre-loaded. Changes are versioned — the client sees the new program on next sync. "Trainer insight" runs a heuristic over the client's last 6 weeks of training.

Screen 33 — Gym operator dashboard



ROLE

When a gym operator signs in, this is home. Three layers: members (who's active, who's churning), equipment (utilization heatmap, broken-machine reports), revenue.

RATIONALE

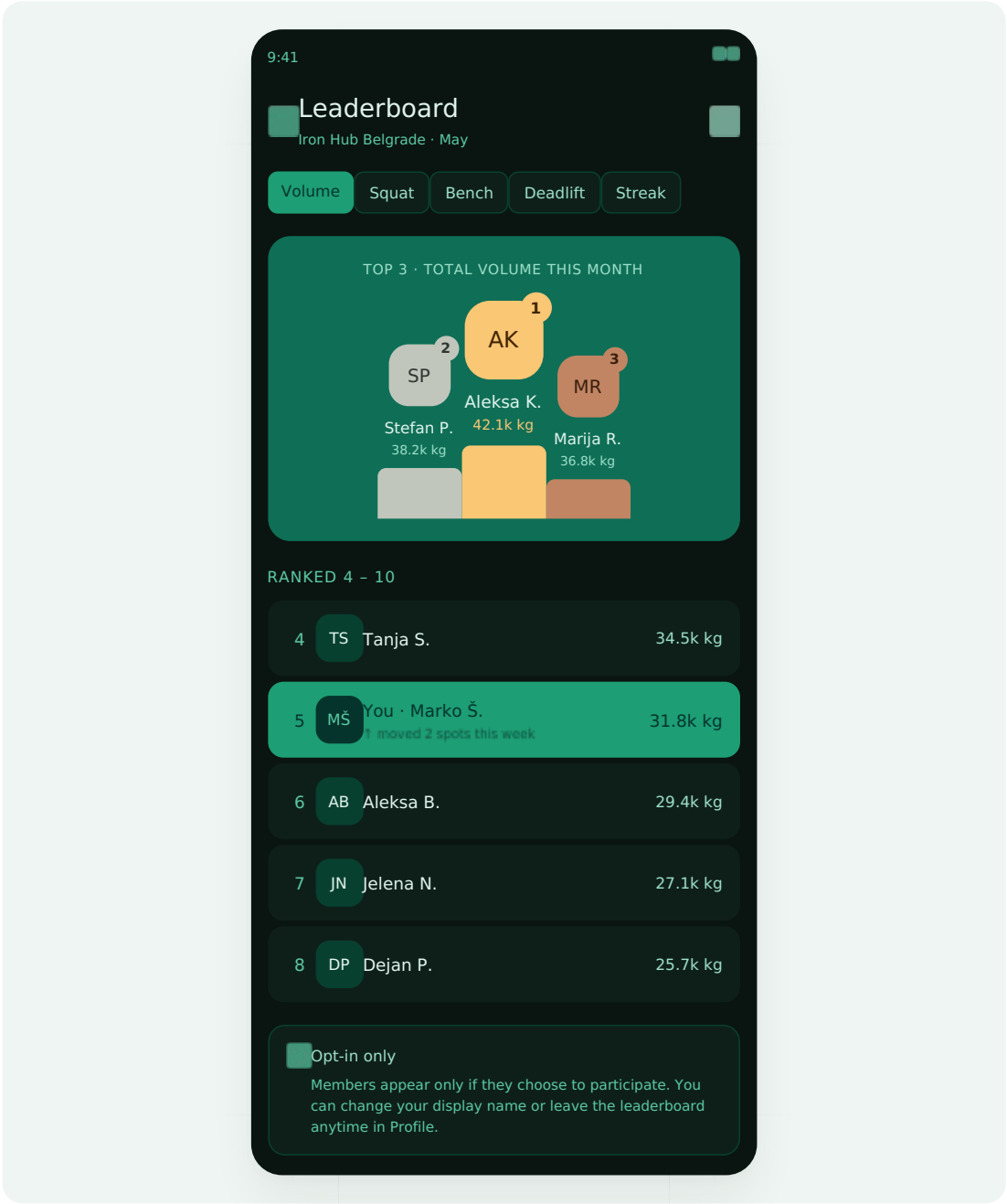
The operator's bottom tab bar is **completely different** — no workout, no library. They're managing a facility, not training. The floor activity histogram is the operator's killer feature — see when peak hours are, staff accordingly. Out-of-service items include duration ("since 4 days") because that's the maintenance KPI.

The privacy disclaimer under membership is **mandatory** — operators see counts, not identities. Aggregate exposure permitted, individual exposure prohibited unless the user explicitly opted into being trained by a gym-employed trainer.

DATA AND BEHAVIOURAL IMPLICATIONS

Floor activity is aggregated from `workout_sessions.started_at` grouped by hour, scoped to sessions at this gym. Anonymous — no user IDs surfaced. Membership churn detection: any user with `last_session_at` > 14 days but with an active membership status. Equipment status comes from `gym_equipment` + `equipment_reports` .

Screen 34 — Gym leaderboard



ROLE

Premium gym feature. When a gym enables it, members can opt into a non-identifying leaderboard. They appear under a chosen display name and their PRs and volume are ranked

against other opted-in members.

RATIONALE

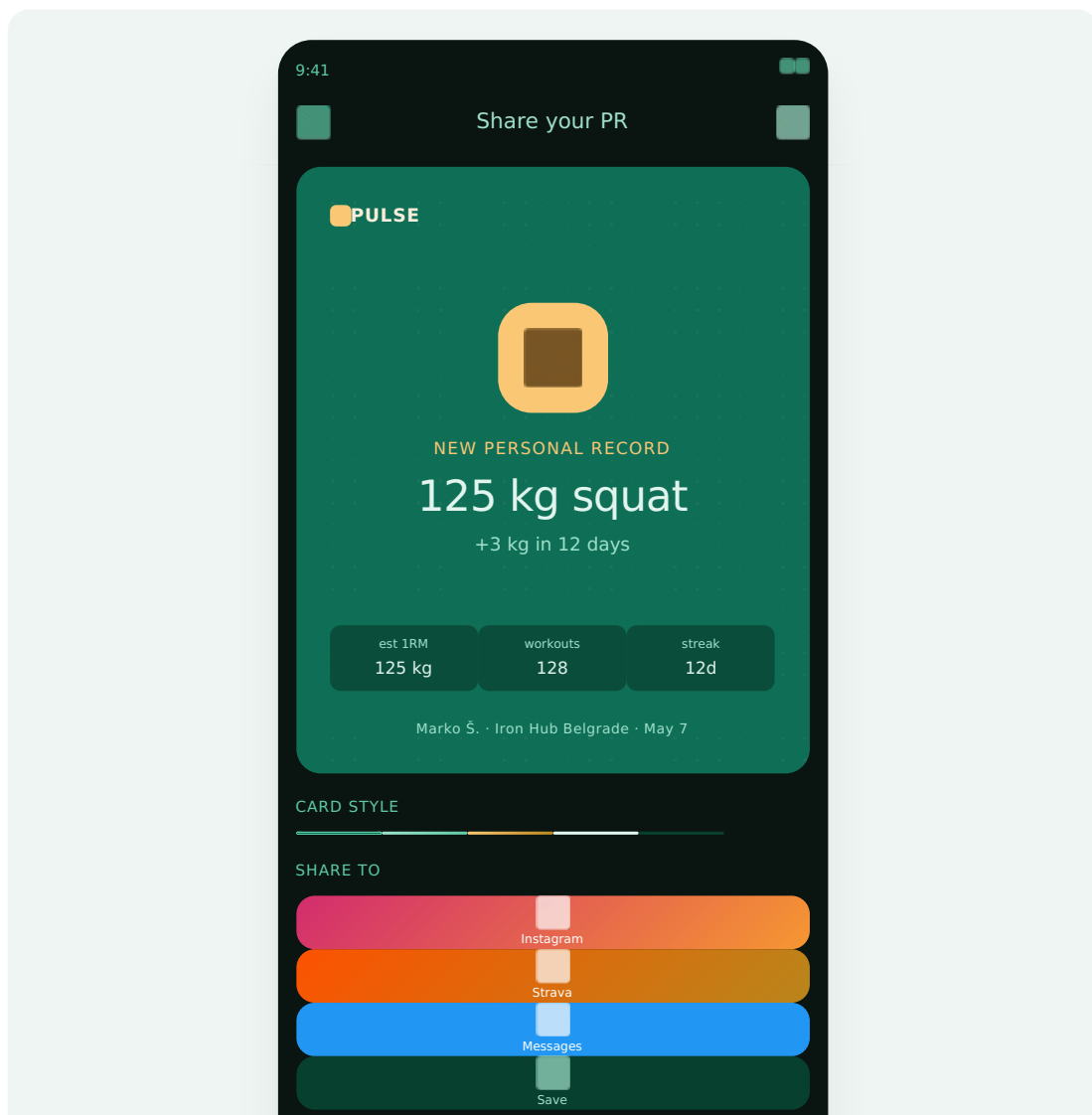
The podium uses gold/silver/bronze color semantics. #1 spot is visually larger so the eye lands on it first. The user's own row is rendered in the saturated green — the user finds themselves instantly. The "moved 2 spots" subtitle turns rank into a progression metric, not just a static position.

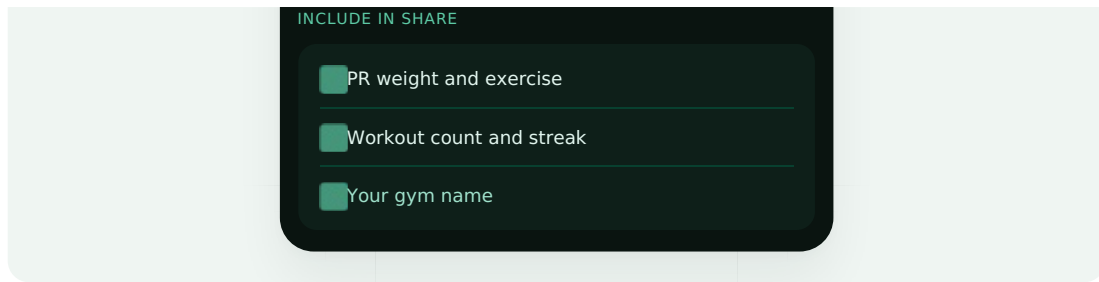
The privacy disclaimer is mandatory — opt-in by default, leave anytime, display name controlled by the user. This is GDPR Article 6 compliance — no profiling without consent.

DATA AND BEHAVIOURAL IMPLICATIONS

Leaderboard participation is opt-in per gym. A `leaderboard_participants` table holds (gym_id, user_id, display_name, opted_in_at). Aggregations run nightly server-side. The leaderboard is a **shareable** entity in our `visibility` schema — gym-scoped (`visibility = 'gym'`) with `is_shareable = true` . Outside the gym context, it's invisible.

Screen 35 — Share or PR moment





ROLE

The "Share" button from PR notifications and from workout completion opens this modal. The user creates a celebration card to share to Instagram, Strava, or save to camera roll.

RATIONALE

The 4:5 portrait card is Instagram and TikTok's preferred aspect ratio. Five style swatches let users match their personal aesthetic. The "Include in share" panel is the privacy control. Every shareable artifact has user-controlled granularity, matching the visibility seam we baked into the schema. Gym name is opt-in because some users don't want to advertise where they train.

DATA AND BEHAVIOURAL IMPLICATIONS

The share card is rendered as a high-DPI image via Skia ([@shopify/react-native-skia](#)) at the moment of share. The render runs locally and produces a PNG ready for the platform share sheet. Each share generates a `share_event` row with `subject_type`, `subject_id`, `platform`, `included_fields`. The user's display name on the card uses the abbreviated form ("Marko Š.") rather than full name unless the user has opted into full-name display.

Data model implications

The screens together imply a coherent data model. Capturing it explicitly here so engineering has a single reference. Tables are described in their functional groupings; not every column is listed, only those that matter for the UX.

Identity and tenancy

`users` — the core user record. Linked 1:1 to a Clerk user via `clerk_user_id`. Holds `user_type` (lifter / trainer / gym_operator), `display_name`, `email`, `current_gym_id`, and a `user_preferences` JSONB blob.

`gyms` — facilities. Holds `name`, `slug`, `coordinates`, `address`, `operator_user_id`, plus an `is_active` flag. Created at QR-sticker generation time by the operator.

`user_gyms` — many-to-many. Each row records when a user first visited a gym and their visit count.

`trainer_clients` — many-to-many between trainer and lifter users. Enforced via Clerk Organizations (trainer is org owner, clients are org members).

Catalog (read-only public data)

`exercises` — the public exercise catalog from ExerciseDB. ~5000 exercises with `name`, `primary_muscle`, `equipment_category`, `video_id` (Mux playback ID), `instructions` (JSONB array of

steps), `default_weight_increment`, `default_increment_unit`, `primary_metric` (weight | reps | time).

`exercise_substitutions` — many-to-many for pre-flight swap suggestions. Each row carries a `similarity_score` and a `reason_label` ("same muscle", "similar pattern").

Per-gym equipment

`gym_equipment` — equipment at a specific gym. Each row is one piece of equipment with `gym_id`, `equipment_category`, `display_label` (e.g., "Bench 3"), `position_index`, `status` (operational / maintenance / out_of_service).

`gym_equipment_pin_mapping` — for selectorized machines. Maps `pin_position` → `approx_kg` per equipment instance.

`equipment_reports` — user-submitted issues. `gym_id`, `reported_by_user_id`, `equipment_name_text`, `note`, `status` (pending / acknowledged / resolved).

Routines and programs

`routines` — user-created or trainer-assigned routines. `user_id`, `name`, `description`, `is_template`, `visibility` (private / trainer_visible / public).

`routine_exercises` — order-positioned exercises within a routine. `routine_id`, `exercise_id`, `position`, `target_sets`, `target_reps`, `target_rest_seconds`, optional `superset_group_id`.

`programs` — multi-week structured plans assigned by trainers to clients. `trainer_id`, `client_id`, `name`, `weeks_total`, `current_week`, `routines_per_week`.

Training data

`workout_sessions` — one row per workout. `user_id`, `gym_id`, `routine_id` (nullable for empty workouts), `started_at`, `ended_at`, `subjective_effort` (1-4 from emoji), `notes`, `entry_source` (manual / qr_scan / scheduled).

`workout_sets` — one row per logged set. `workout_session_id`, `exercise_id`, `gym_equipment_instance_id`, `weight_kg`, `pin_position`, `reps`, `rpe_inferred`, `is_warmup`, `is_failure`, `set_group_id`, `set_group_position`, `created_at`.

`set_groups` — superset and drop set linkage. `workout_session_id`, `kind` (superset / drop_set / giant_set / circuit), `rounds` (for supersets), `rest_seconds`.

`personal_records` — derived from set commits. `user_id`, `exercise_id`, `gym_equipment_instance_id`, `record_type` (est_1rm / max_weight / max_reps), `value`, `achieved_at`, `previous_value`.

Body composition

`body_measurements` — `user_id`, `recorded_at`, `weight_kg`, `body_fat_percent`, `lean_mass_kg`.

`body_circumference` — `user_id`, `site` (chest / arm / waist / thigh / custom), `value_cm`, `recorded_at`.

`body_photos` — `user_id`, `storage_path`, `recorded_at`, optional `pose_type`.

Preferences and overrides

`user_preferences` — JSONB on the user row. Includes `units` (kg/lb), `rest_seconds_default`, `haptics_enabled`, `notifications`, and various has-seen-hint flags.

`user_exercise_preferences` — per-user per-exercise overrides. `user_id`, `exercise_id`, `default_weight_increment`, `default_increment_unit`.

Subscriptions, integrations, messaging

`user_subscriptions` — server-mirrored RevenueCat entitlements. `user_id`, `tier` (free / trial / premium), `expires_at`, `platform`.

`health_integrations` — per-user integration state. `user_id`, `provider` (apple_health / health_connect), `connected_at`, `last_sync_at`, `permissions_granted` (JSONB), `duplicates_skipped` count.

`notifications` — `user_id`, `kind`, `actor_id`, `subject_id`, `payload` JSONB, `read_at`.

`messages` — trainer-client async chat. `thread_id`, `sender_id`, `recipient_id`, `body`, `attachments`.

Sharing

`leaderboard_participants` — opt-in leaderboard membership. `gym_id`, `user_id`, `display_name`, `opted_in_at`.

`share_events` — analytics on shares. `user_id`, `subject_type`, `subject_id`, `platform`, `included_fields`.

Visibility seams

Every shareable entity carries three columns, baked into the schema from v1 even though most are not yet exposed in the UI:

- `visibility` — enum: private / trainer_visible / gym_visible / public. Default: private.
- `is_shareable` — boolean. Default: false. Even if visibility allows it, the user must explicitly mark something shareable.
- `origin_id` — uuid. If this entity was copied from another user (e.g., a routine cloned from a trainer's template), this points back to the source.

These three columns are present on `routines`, `personal_records`, `workout_sessions`, `body_measurements`, and `body_photos`. They let us expand sharing surfaces post-v1 without schema migrations.

Behavioural rules baked into the design

These are rules that should be enforced in the codebase via the agent files and through code review.

Friction budgets

- Log a set with same weight/ reps as last set: **1 tap** (the primary "Log 80 × 8" button).
- Log a set with adjusted weight or reps by one increment: **2 taps** (one step button, one log button).
- Add an exercise mid-workout: **3 taps** (open selector, choose exercise, confirm with the green + button).
- Start a planned workout from cold launch: **2 taps** (open app, tap "Start workout" hero button).
- Switch current gym: **3 taps** (Profile, My gyms, Switch on the target gym).

Implicit-confirm rule

Every set is provisional until the user either taps "Log" or navigates within the same exercise. Navigating to a different exercise without confirming silently discards the set with a small undo affordance. No "are you sure?" modals.

Single accent rule

Each screen has at most one element rendered in the saturated `#1D9E75` accent. Eyes are trained, screen after screen, to find the next action in this colour.

Disclaimer rule

Every screen that displays body composition data (body fat percent, lean mass, BMI) must include the short disclaimer card: "For personal fitness tracking only. Not a medical assessment. Consult a healthcare professional for medical guidance." This is non-optional and is enforced by the fitness-domain-expert agent.

Privacy default rule

Every user-generated entity defaults to private visibility. Sharing requires explicit user action. Even when shared, the user controls granularity (gym name on/off, full name vs abbreviated, etc.).

Two-speed data architecture

Personal data flows through PowerSync local-first. Aggregate or feed data (leaderboards, gym dashboards) flows through server-paginated TanStack Query — never PowerSync. PowerSync's strength is offline-correct personal data; it's the wrong tool for unbounded aggregate streams.

Optimistic UI rule

Every logged set, every confirmed routine save, every measurement entry commits to local SQLite immediately and reflects in the UI before the network sync completes. Network sync is invisible to the user and happens in the background. Failed syncs are retried with exponential backoff; persistent failures surface as a small "syncing" indicator in the header but do not block UI.

Wall-clock-anchored timer rule

Every rest timer uses wall-clock anchoring (compute remaining from `start_at + target_seconds - now`), not JS interval decrement. This ensures timers survive app backgrounding, phone calls, and lock screens.

Patch 8 — what the agent files need to know

The screen designs in this catalog imply specific updates to the existing agent files in `.claude/agents/`. Capturing these here as a "patch 8" so the changes can be applied to the agent files in one batch.

data-sync-engineer

Add to the schema documentation:

- `set_group` table with kind enum (superset / giant_set / circuit / drop_set) and `rounds` , `rest_seconds` fields. Sets within a group carry `set_group_id` and `set_group_position` .
- `gym_equipment_instance_id` on `workout_sets` . This is the per-machine binding.
- `user_exercise_preferences` table for per-user per-exercise increment overrides.
- Visibility seam columns (`visibility` , `is_shareable` , `origin_id`) on every shareable entity.

Add to the sync rules test scenarios:

- Test 11: Superset round atomicity — both A and B sets must sync atomically.
- Test 12: Drop set group integrity — top set + drops must sync atomically.
- Test 13: Per-equipment PR scoping — PR detection runs per `gym_equipment_instance_id`.

ux-flow-architect

Add the implicit-confirm pattern as a documented rule. Friction budgets:

- "Log same as last": 1 tap
- "Log adjusted by one increment": 2 taps
- "Add exercise mid-workout": 3 taps

Add the stepped haptic numeric input pattern — this replaces the system keyboard for all in-workout numeric input.

fitness-domain-expert

Add the per-exercise default-increment rules:

- Barbell compound: ± 5 kg
- Selectorized machine: ± 1 pin
- Cable kg-stack: ± 5 kg
- Isolation barbell: ± 2.5 kg
- Dumbbell: ± 2.5 kg
- Bodyweight+load: ± 1.25 kg

Add the body composition disclaimer rule as mandatory wherever body comp surfaces appear. Add the App Store positioning: Health & Fitness, not Medical. MHMDA / GDPR / CCPA compliance required; HIPAA does not apply.

mobile-architect

Add the pre-cache routine for offline pattern. Premium routines flagged for offline are pre-fetched on tap, including Mux video files via `expo-video-cache` .

Add the Live Activity behaviour for rest timer: created on "Log set" with `target_seconds` from the routine, updated every 5 seconds via `expo-live-activity.update()` , dismissed on "Start next set" or timer expiration, interactive buttons (skip, +30s) supported via iOS 17+ Live Activity API.

qr-and-deeplink-specialist

Add the optimistic prefill rule for QR landing — by the time the user lands on the smart logger, weight

+ reps + equipment binding are pre-filled from the local database, with the JWT signature verification happening in parallel.

note-keeper

Add an override-logging mechanism. When users override a default (per-exercise increment, default rest, etc.), the note-keeper logs the override with timestamp + previous value + new value for future analytics.

What's next

This document is the complete UI/UX specification for the v1 Pulse app. With it, the path forward is:

1. **Convert to Figma.** A real designer takes this spec and builds it in Figma at production fidelity, creating the component library, the responsive variants, the dark/light mode pairs, and the asset exports for engineering. The spec gives them the decisions; they handle the pixel-pushing.
2. **Apply Patch 8 to the agent files.** The schema changes, the friction budgets, the per-exercise increment rules, and the implicit-confirm pattern need to be folded into `.claude/agents/data-sync-engineer.md`, `ux-flow-architect.md`, `fitness-domain-expert.md`, `mobile-architect.md`, `qr-and-deeplink-specialist.md`, and `note-keeper.md`.
3. **Implement in slices.** Start with the lifter journey from onboarding through one logged set. That's screens 1, 2, 7, 8, 9, 10, 11, and the post-workout summary from screen 3. Roughly two-week implementation slice for a competent React Native developer with the agent files loaded.
4. **Iterate with real users.** The implicit-confirm pattern and the smart logger are the highest-risk innovations. Get them in front of 10 users before building out the analytics surface. If they don't land, the rest of the app loses its differentiator.
5. **Sequence the rest.** Progress dashboard, history, body measurements come next (Batch 2 in this spec). Trainer surfaces (Batch 4) come last because the lifter app has to be excellent before trainers will trust their clients to it.

The product is designed for sub-100ms set logging, offline-correct sync, and a single emotional principle: respect the user's time and intelligence.

END OF SPECIFICATION

Pulse

Design specification · v1.0 · May 2026